

経済学のためのデータ分析入門 (Python 編)

そら

2026-02-01

Table of contents

はじめに	11
シラバス	11
準備	12
第1回 Python と Jupyter (/VS Code) の基礎	13
なぜ今、Python を学ぶのか	13
科学的検証のためのツール	14
Python と Excel：対立ではなく使い分け	14
Jupyter (/VS Code) という「コックピット」	15
Python と対話する（計算と変数）	15
情報を箱に入れる：変数	16
便利な道具：関数	16
「エラー」は Python からの返信	17
グラフを描いてみる	17
課題	18
第2回 データの構造と扱い	19
データを「整える」思想	19
pandas：人間中心の Python	20
データフレーム：世界を表にする	20
現実を Python に取り込む（CSV 読み込み）	20
データ操作の文法（pandas）	22
つなげて書く：メソッドチェーン	22
1. 切り分ける：列の選択と行の抽出	22

2. 加工する：新しい列を作る	23
3. 並べる：並べ替え	23
4. 要約する：グループ集計	24
ストーリーで書く	25
データ操作の補足事項	26
列の選び方	26
データのチラ見（可視化）	26
課題	27
練習問題	27
練習 1: 基礎確認	27
練習 2: 仮説の検証	28
練習 3: パイプラインの構築	28
第 3 回 記述統計 (1) 数値要約	29
情報を「縮約」する	29
準備	30
中心を探す：代表値	30
1. 全員の声を聞く：平均値 (Mean)	30
2. 真ん中の人を見る：中央値 (Median)	31
3. 多派を見る：最頻値 (Mode)	31
意思決定：平均か、中央値か	31
広がりを知る：散布度	32
最大と最小：範囲 (Range)	32
中心からの距離：分散 (Variance)	32
単位を戻す：標準偏差 (Standard Deviation)	33
単位を消す：変動係数 (CV)	33
全体像をスキャンする：四分位数	34
安定した広がり：四分位範囲 (IQR)	34
ノイズか、シグナルか：外れ値の検出	35
分布の歪みを可視化する	36
グループごとに要約する	37
課題	38
練習問題	38

練習 1: 基本的な記述統計の算出	38
練習 2: グループ別の比較検証	38
練習 3: 外れ値の影響を体感する	38
第 4 回 記述統計 (2) データの可視化	41
要約の限界と「アンスコム警鐘」	41
準備	43
キャンバスに層を重ねる: seaborn / matplotlib の考え方	44
1. 分布の形状を見る	44
山の形を知る: ヒストグラム	44
構造を要約する: 箱ひげ図	45
分布の機微を見る: バイオリンプロット	46
2. 関係性を探る	47
2 つの量の相関: 散布図	47
状況ごとの違い: ファセット分割	49
グラフを「語らせる」ための調整	50
統計情報の重ね書き	50
ユニバーサルデザインへの配慮	51
課題	51
練習問題	52
練習 1: ヒストグラムの最適化	52
練習 2: 多変量の可視化	52
練習 3: 散布図への情報付加	52
第 5 回 確率論の基礎	53
不確実性を飼いならす言語	53
可能性の地図: 確率分布	54
デジタルとアナログ: 離散と連続	54
離散型の実験: サイコロの地図	54
分布の特徴を捉える	55
1. 期待値 (Expected Value): 「賭け」の相場	55
2. 分散 (Variance): リスクの大きさ	56
線形性: 計算を楽にする魔法	56

連続の世界へ	57
確率密度関数とは	57
実験：一様分布シミュレーション	57
同時確率と独立性	58
独立であるとはどういうことか	58
課題/練習問題	59
課題 1: 歪んだコインのギャンブル	59
課題 2: 独立性の検証	59
第 6 回 主要な確率分布	61
世界を支配する「釣り鐘」	61
1. 統計学の王様：正規分布	62
4 つの基本操作	62
正規分布を描いてみる	62
パラメータを変えてみる	63
現実の問いに答える：確率計算	64
68-95-99.7 ルール	65
2. コイン投げの積み重ね：二項分布	66
二項分布の 3 条件	66
シミュレーション：運命の分かれ道	66
回数を増やすと、奇跡が起きる	68
3. 正規分布の子供たち（派生分布）	69
カイ二乗分布：ブレの分析官	70
t 分布：現場の代役	70
F 分布：比較のスペシャリスト	70
課題	70
練習問題	71
練習 1: テストの点数分布	71
練習 2: 不良品の発生確率	71
練習 3: 乱数シミュレーション	71
第 7 回 標本分布	73
一滴の血液で、全身を知る	73

1. 未知の世界と鍵穴：母集団と標本	74
全数調査の限界	74
ランダムという名の公平性	74
2. パラレルワールドの実験：標本分布	74
3. 数学的な魔法	75
大数の法則：真実への収束	75
中心極限定理 (CLT)：混沌からの秩序	77
サンプルサイズと分布の鋭さ	78
4. 信用の尺度：標準誤差	80
標準偏差 (SD) vs 標準誤差 (SE)	80
課題	81
練習問題	81
練習 1: 指数分布の CLT	81
練習 2: サンプルサイズと精度	81
練習 3: 記述統計と推測統計	82
まとめ	82

第 8 回 統計的推定 83

「だいたい」を科学する	83
1. 推定の基礎：レシピと料理	84
良いレシピの条件	84
2. 点推定：一点豪華主義	84
3. 区間推定：輪投げのゲーム	85
95% 信頼区間の本当の意味	85
4. t 分布：不確実性のペナルティ	87
Python で実践：平均の区間推定	88
信頼区間の幅を決める 3 要素	88
5. 母比率の区間推定	89
課題	90
練習問題	90
練習 1: 信頼区間の幅とサンプルサイズ	90
練習 2: 母比率のシミュレーション	90
練習 3: 記述統計と推測統計	91

まとめ	91
第 9 回 仮説検定 (1)	93
裁判官になるための統計学	93
1. 裁判の基本ルール：疑わしきは罰せず	94
登場人物（仮説）	94
2. 証拠の評価：p 値という驚き	94
判決の基準：有意水準	95
3. シミュレーション：無罪の証明	95
4. 実践：1 標本の t 検定	96
5. 冤罪のリスク：2 つの過誤	97
課題	98
練習問題	98
練習 1: 結論の解釈	98
練習 2: サンプルサイズと有意差	98
練習 3: 誤りの種類	98
まとめ	98
第 10 回 仮説検定 (2)	101
A vs B：決着をつけよう	101
1. 2 つの島を比べる：対応のない t 検定	102
地形を選ばない万能車：Welch の t 検定	102
2. ビフォー・アフター：対応のある t 検定	103
変化を捕まえる	104
3. その差は、どれくらい凄いのか？：効果量	104
シグナルの強さを測る	105
課題	106
練習問題	106
練習 1: 検定の選択	106
練習 2: サンプルサイズと p 値	106
練習 3: 等分散性の確認はいらない？	106
まとめ	106
第 11 回 相関と単回帰分析	109

未来を予測する魔法	109
1. 2 人のダンス：相関 (Correlation)	110
実践: 支払額とチップの関係	110
2. 予測マシンを作る：単回帰分析	111
交換レートとしての「傾き」	111
直線の可視化	112
3. モデルの品質チェック：残差診断	113
決定係数: 説明力	113
残差の健康診断	114
課題	115
練習問題	115
練習 1: 相関の罫	115
練習 2: 決定係数の解釈	115
練習 3: 残差のパターン	116
まとめ	116
第 12 回 重回帰分析と因果推論	117
複雑な現実を解き明かす	117
1. 複数のツマミを操る：重回帰分析	118
「他を固定して」見る技術 (Ceteris Paribus)	118
実践: 支払いと人数の分離	118
2. 実践的モデリング	120
ダミー変数：スイッチの ON/OFF	120
交互作用：「相乗効果」に気づく	120
3. 真犯人を探せ：因果推論とバイアス	122
典型例：教育と賃金のミステリー	122
シミュレーション：バイアスの除去	122
4. 良いモデルの選び方	123
多重共線性（マルチコ）：似た者同士の喧嘩	123
調整済み決定係数：シンプルさへのボーナス	124
係数の可視化	124
最終課題	126
講義のまとめ	127

練習問題	127
----------------	-----

はじめに

この講義ノートは、経済学部 の 1 年生を対象とした「Python による統計学入門」の資料です。プログラミング未経験者を想定し、データの読み込みから基礎的な統計分析、そして回帰分析までを半期（全 12 回）で学びます。

本講義では、Python の **pandas**（データ操作）と **seaborn / matplotlib**（可視化）を中心に解説します。

シラバス

1. Python と Jupyter (/VS Code) の基礎: インストール、基本操作、計算
2. データの構造と扱い: ベクトル、データフレーム、データの読み込み
3. 記述統計 (1) 数値要約: 平均、分散、標準偏差、四分位数
4. 記述統計 (2) データの可視化: seaborn / matplotlib によるグラフ作成
5. 確率論の基礎: 確率変数、期待値、分散
6. 主要な確率分布: 正規分布、二項分布
7. 標本分布: 母集団と標本、中心極限定理
8. 統計的推定: 点推定、区間推定
9. 仮説検定 (1): 検定の考え方、1 標本の t 検定
10. 仮説検定 (2): 2 標本の平均の差の検定
11. 相関と単回帰分析: 散布図、相関係数、最小二乗法
12. 重回帰分析と因果推論: 複数の説明変数、因果関係と相関関係

準備

この講義では以下のソフトウェアを使用します。

- [Python](#)
- [JupyterLab](#)（または VS Code）

また、以下のパッケージを主に使用します。

```
pip install -r requirements.txt
```


第1回 Python と Jupyter (/VS Code) の基礎

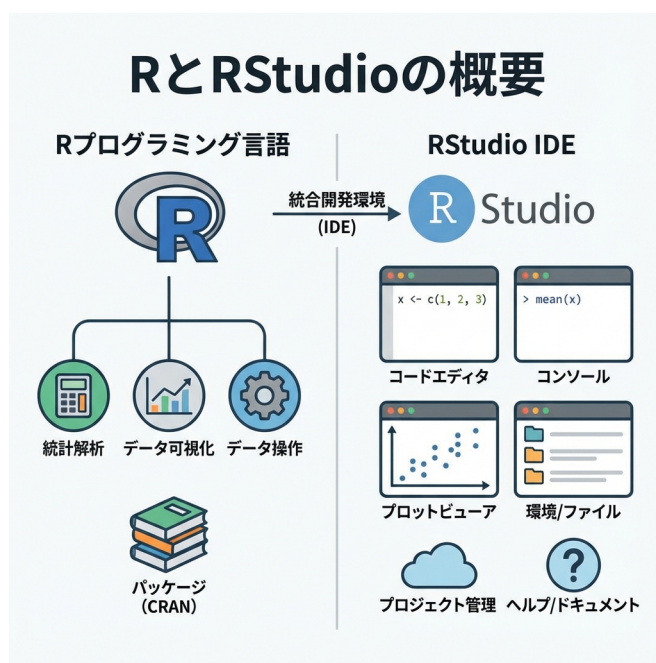


Figure1: Python と Jupyter (/VS Code) の基礎

なぜ今、Python を学ぶのか

統計解析ソフトは山ほどあります。Excel で事足りることもあれば、Python がハマる場面もあるでしょう。それでも、多くの研究者が **Python** を選び続けるのには理由があります。

「科学的な正しさ」への執着。これに尽きます。

本講義の狙いは、単なるツールの操作説明ではありません。**Python** による「再現性 (Reproducibility)」の思想を理解し、**Jupyter (または VS Code)** を通じてそれを実践する力を養うことです。

科学的検証のためのツール

なぜこれほど Python を推すのか。最大の理由は**再現性が確保できるから**です。Excel での分析は、どうしても「手作業」の連続になります。どのセルをコピーし、どこでフィルタをかけ、どのグラフを選んだか。その履歴を完全に追跡するのは困難です。一方、Python はすべての操作をコードとして残します。コードは「手順書」そのもの。第三者が同じコードを実行すれば、間違いなく同じ結果が出ます。科学的な検証において、この特性は欠かせません。

もちろん、実利的なメリットも大きいです。

- **OSS とコスト**: 個人・組織を問わず無料 (GPL ライセンス)。
- **ライブラリの豊富さ**: 数値計算 (NumPy)、データ操作 (pandas)、可視化 (matplotlib / seaborn)、統計モデリング (statsmodels) などが揃っています。
- **キャリア**: データ分析・データサイエンスの職務記述書 (JD) で Python は定番スキルです。

(Web アプリ開発やシステムへの組み込みなら Python や C++ が適しています。適材適所です)

Python と Excel : 対立ではなく使い分け

よく「R か Excel か」と二項対立で語られますが、実際は「併用」が正解です。

ツール	得意な領域	苦手な領域
Excel	直感的な操作、データの閲覧、小規模な集計	大規模データ (100 万行～)、手順の記録、複雑な統計モデル

ツール	得意な領域	苦手な領域
Python	大規模データの処理、高度な統計解析、再現性のある描画	データの直接入力、閲覧（スプレッドシート的な操作）

私だって、データの概観やちょっとした検算には Excel を使います。でも、論文に載せる分析や、来月も同じ処理をする業務なら、迷わず Python を選びます。

Jupyter (/VS Code) という「コックピット」

Python 本体はただの「計算エンジン」です。人間が楽に扱うために、ノートブック環境の **Jupyter**（または統合開発環境の VS Code）を使います。画面は 4 分割されていますが、データの流れて考えると分かりやすいでしょう。

1. **Editor / Notebook**：ここで「指示書（コード）」を書く。
 - 料理ならレシピを書く場所。ここに書いたものだけが「再現可能な記録」として残ります。
2. **実行結果（セル出力 / Terminal）**：指示を実行し、結果を受け取る。
 - 実際に Python が計算する場所。ちょっとした確認用です。
3. **変数/データの確認**：データフレームや変数を確認する。
 - いまメモリ上に何があるかを把握します。
4. **Plots / Files**：グラフやファイルを確認する。
 - 描かれた図や保存物はここで見ます。

基本は、「左上で書いて、左下で実行させて、右と右下で結果を確認する」。この繰り返しです。

Python と対話する（計算と変数）

まずは難しく考えず、Python を「やたら高機能な電卓」として使ってみましょう。セル（または REPL）に直接入力してみてください。

```
1 + 1
(100 - 20) / 4
```

20.0

情報を箱に入れる：変数

計算結果を使い捨てにせず、取っておくための箱が**変数（オブジェクト）**です。Python では=（代入）を使います。「右の値を左の箱に入れる」イメージです。

```
price = 1500
amount = 3
total = price * amount
total
```

4500

大事なのは、price や amount という「名前」でデータを管理できること。「1500」という数値の意味を、人間が覚えておく必要はありません。

便利な道具：関数

よく使う処理は **関数**として用意されています。「機能名（データ）」の形で呼び出します。

```
import statistics as stats

# 数値のリスト（列）を作る
scores = [80, 90, 75, 60, 95]

# 平均値を計算する
stats.mean(scores)
```

80

関数は無数にありますが、最初は「やりたいこと（機能）」と「入れるもの（引数）」の対

応さえ意識できれば十分です。

「エラー」は Python からの返信

プログラミング初心者にとって、赤い文字のエラーメッセージは怖いかもしれません。「お前は間違ってる」と怒られた気分になるからでしょう。でも、誤解です。エラーは Python からの「この指示だと、ここが分からなくて実行できません」という、ただの返信です。

- **NameError** : 「その名前が見当たりません」(変数を作る前に使っていませんか?)
- **ModuleNotFoundError** : 「そのライブラリが見つかりません」(インストールし忘れていませんか?)

エラーが出たら、まずはメッセージを読んでみてください。解決のヒントは必ずそこにあります。

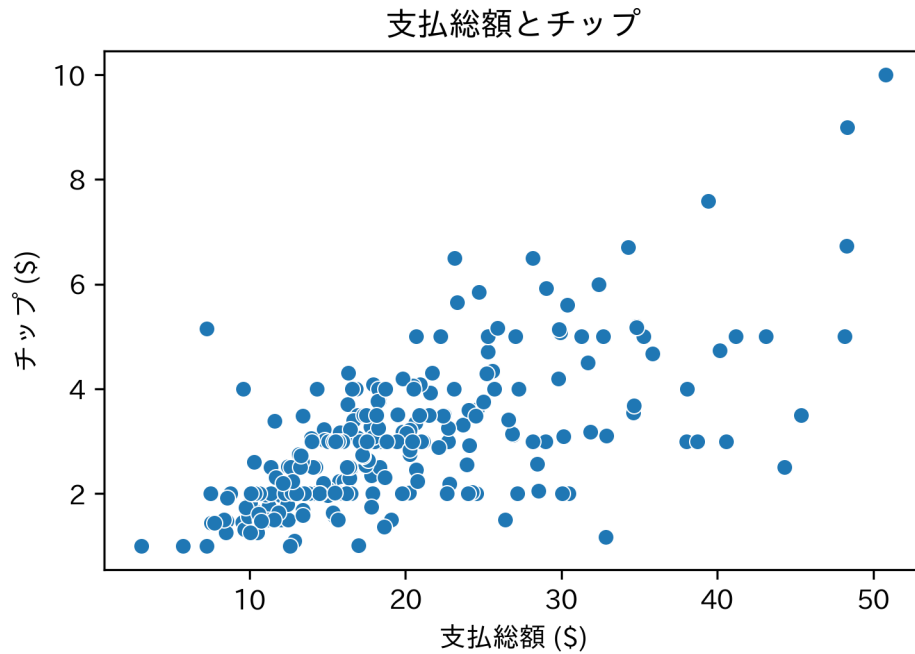
グラフを描いてみる

最後に、Python の「華」、可視化を試してみます。解説は省きますが、数行書くだけで美しい散布図が描けることを確認してください。

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)

ax = sns.scatterplot(data=tips, x="total_bill", y="tip")
ax.set(title="支払総額とチップ", xlabel="支払総額 ($)", ylabel="チップ ($)")
plt.show()
```



このグラフも、コードの数値を少し変えるだけで、明日になっても（あるいは10年後でも）全く同じものを再現できます。

課題

1. 自身の年齢を変数 `age` に代入してください。
2. 変数 `age` を使って、10年後の年齢を計算してください。
3. 任意の数値を3つ格納したリストを作成し、その平均値を求めてください。

第 2 回データの構造と扱い

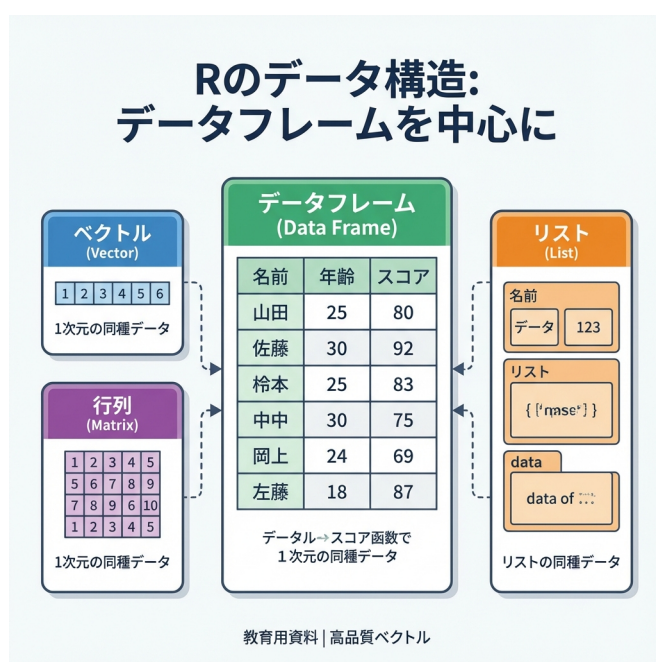


Figure2: データの構造

データを「整える」思想

データ分析において、一番時間を食う作業は何でしょうか。複雑な統計モデルの構築ではありません。「データの掃除 (Data Wrangling)」です。形式の揃っていないデータ、謎の空白、表記の揺れ。これらを人間が扱える形に整えるだけで、プロジェクトの時間の 8 割が消えることすらあります。

本節では、この泥臭い作業をエレガントに片付ける武器、**pandas** を導入します。

pandas：人間中心の Python

Python にもデータを扱う手段は色々ありますが、表形式データ（行と列）を扱うなら **pandas** が事実上の標準です。「人間が読みやすく、書きやすい」ことを優先しつつ、現実のデータ分析に必要な機能が一通り揃っています。

1. インストール（世界への入り口）：初回のみ

```
pip install pandas
```

2. ロード（工具箱を開ける）：起動するたびに必要

```
import pandas as pd
```

これだけで、必要な道具一式（加工、可視化、読み込み...）は揃います。

データフレーム：世界を表にする

現実のデータは、混沌としています。それをコンピュータが理解できる形は、結局「行と列」だけです。Python (pandas) での呼び名も **データフレーム** です。

- 行 (Row)：ひとつの観測単位（1 人の人間、1 回の取引、1 日の記録）
- 列 (Column)：属性や変数（年齢、金額、気温）

現実を Python に取り込む（CSV 読み込み）

手元の Excel や CSV ファイルは、まだ「外」にあります。これを「中」（Python の計算世界）に引き入れる儀式が `read_csv()` です。

今回は、Python のライブラリ等でもよく使われる `tips`（レストランのチップ支払い記録）を Web から直接読み込んでみます。

```
import pandas as pd
```

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
```



```
tips = pd.read_csv(url)
```

```
# データの「顔見せ」
```

```
tips.info()
```

```
tips.head()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 244 entries, 0 to 243
```

```
Data columns (total 7 columns):
```

```
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null       float64
1   tip         244 non-null       float64
2   sex         244 non-null       object
3   smoker      244 non-null       object
4   day         244 non-null       object
5   time        244 non-null       object
6   size        244 non-null       int64
```

```
dtypes: float64(2), int64(1), object(4)
```

```
memory usage: 13.5+ KB
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

読み込んだ瞬間、変数の型（数値なのか、文字なのか）は自動判定されます。Excel によくある「数字なのに文字扱い」みたいなトラブルも、入り口で防げます。

[要一次情報: データセットの詳細な定義書の所在]

データ操作の文法 (pandas)

pandas には、データ加工の基本操作が一通り揃っています。「列を選ぶ」「行を絞る」「新しい列を作る」「並べ替える」「グループ集計する」——これらがデータ分析の基本動作です。

つなげて書く：メソッドチェーン

pandas では「メソッドチェーン（メソッドを連結して書く）」で、思考の順番に処理を書けます。

- 単発: `tips[["total_bill"]].head()` → 「列を選んで、先頭を見る」
- 連結: `tips.query('time == "Dinner"').sort_values("tip", ascending=False).head()` → 「絞って、並べて、先頭を見る」

1. 切り分ける：列の選択と行の抽出

分析はまず、要らない部分を削ぎ落とすことから始まります。

列（変数）を選ぶ「分析に性別と喫煙情報は関係ない」なら、こう。

```
tips[["total_bill", "tip"]].head()
```

	total_bill	tip
0	16.99	1.01
1	10.34	1.66
2	21.01	3.50
3	23.68	3.31
4	24.59	3.61

行（観測）を絞る「ディナーの客だけ見たい」なら、こう。

```
tips.query('time == "Dinner"]').head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

2. 加工する：新しい列を作る

既存の列を使って新しい指標を作るときは `assign()` が便利です。「チップの額」そのものより、「支払額に対するチップの割合」の方が意味があるでしょう。

```
(
    tips
    .assign(tip_rate=lambda df: df["tip"] / df["total_bill"])
    .loc[:, ["total_bill", "tip", "tip_rate"]]
    .head()
)
```

	total_bill	tip	tip_rate
0	16.99	1.01	0.059447
1	10.34	1.66	0.160542
2	21.01	3.50	0.166587
3	23.68	3.31	0.139780
4	24.59	3.61	0.146808

3. 並べる：並べ替え

データは最初、「記録順」です。これを「意味のある順」に変えます。

```
# チップを多く払った順（降順）に並べる
tips.sort_values("tip", ascending=False).head()
```

	total_bill	tip	sex	smoker	day	time	size
170	50.81	10.00	Male	Yes	Sat	Dinner	3
212	48.33	9.00	Male	No	Sat	Dinner	4
23	39.42	7.58	Male	No	Sat	Dinner	4
59	48.27	6.73	Male	No	Sat	Dinner	4
141	34.30	6.70	Male	No	Thur	Lunch	6

4. 要約する：グループ集計

個別の値より「傾向」が知りたいなら、グループにまとめて（`groupby`）、縮約（`agg`）します。「ランチとディナー、どちらが羽振りがいいのか？」という問いに答えてみましょう。

```
(
    tips
    .groupby("time", as_index=False)
    .agg(
        平均支払額=("total_bill", "mean"),
        平均チップ=("tip", "mean"),
        データ数=("tip", "size"),
    )
)
```

	time	平均支払額	平均チップ	データ数
0	Dinner	20.797159	3.102670	176
1	Lunch	17.168676	2.728088	68

ストーリーで書く

これらの「動詞」と「パイプ」を組み合わせれば、複雑なデータ処理も一つのストーリーとして記述できます。

問い：土曜日のディナーにきた客のうち、特にチップを弾んでくれた（総額の 20% 以上）のはどんな人たちか？ 支払額が高い順に見たい。

この思考プロセスをそのままコードにします。

```
(
    tips
    .query('day == "Sat" and time == "Dinner"')
    .assign(rate=lambda df: df["tip"] / df["total_bill"])
    .query("rate >= 0.2")
    .sort_values("total_bill", ascending=False)
    .loc[:, ["sex", "size", "total_bill", "tip", "rate"]]
)
```

土曜日のディナー客で
チップ率を計算し
20% 以上に絞り込み
支払額が高い順に並べ替え
必要な情報だけ見る

	sex	size	total_bill	tip	rate
239	Male	3	29.03	5.92	0.203927
214	Female	3	28.17	6.50	0.230742
63	Male	4	18.29	3.76	0.205577
108	Male	2	18.24	3.76	0.206140
20	Male	2	17.92	4.08	0.227679
109	Female	2	14.31	4.00	0.279525
110	Male	2	14.00	3.00	0.214286
228	Male	2	13.28	2.72	0.204819
232	Male	2	11.61	3.39	0.291990
67	Female	1	3.07	1.00	0.325733

この「読みやすさ」こそ、私たちが pandas（メソッドチェーン）を使う理由です。数ヶ月後の自分が見ても、「何がしたかったか」一目で分かります。

データ操作の補足事項

列の選び方

pandas にも「取り出し方の違い」があります。

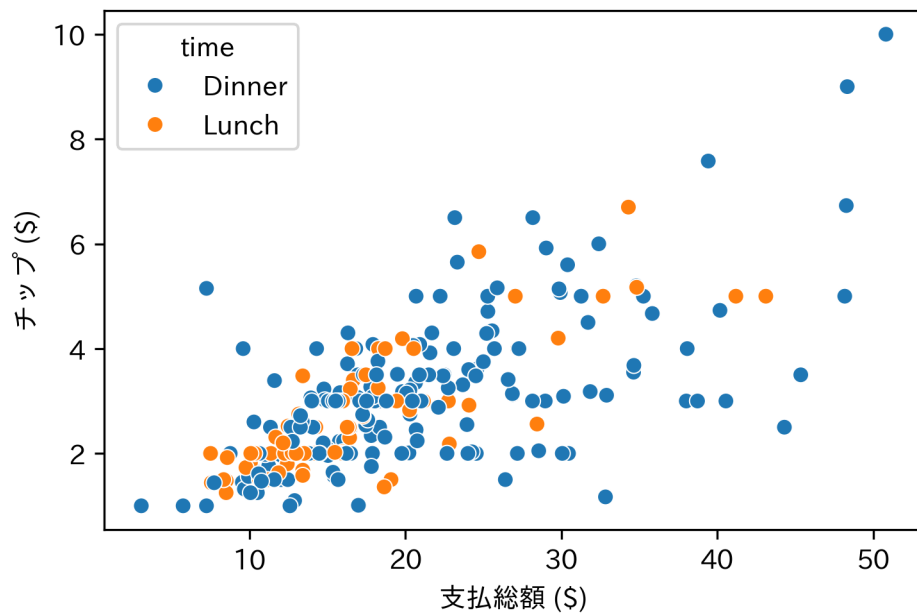
- `tips["total_bill"]` : 1 本の列 (Series) として取り出す。計算式の内部で使うことが多い。
- `tips[["total_bill"]]` : データフレームのまま取り出す。加工を続けるときに便利。使い分けですが、複数列を扱う処理では `tips[["col1", "col2"]]` の形が無難です。

データのチラ見 (可視化)

数字だけだと見逃します。seaborn でサッと可視化する癖をつけてください。

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

ax = sns.scatterplot(data=tips, x="total_bill", y="tip", hue="time")
ax.set(xlabel="支払総額 ($)", ylabel="チップ ($)")
plt.show()
```



課題

自分でストーリーを作ってみましょう。

1. `day` が “Sun” (日曜日) である行のみを抽出する (`filter`)。
2. 抽出したデータから、`total_bill` と `size` の 2 列のみを選択して表示する (`select`)。

練習問題

正解かどうかより、「コードが思考の流れを表しているか」を意識してください。

練習 1: 基礎確認

`glimpse()` や `dim()` を使い、このデータセットが「何行何列」で、「変数の型は何か」を言葉で説明してください。

練習 2: 仮説の検証

「女性客の方がチップ率が高いのではないか？」という仮説を検証するコードを書いてください。（ヒント：`filter` で女性を抽出し、`mutate` で率を計算し、`summarize` で平均を出す、など方法は幾通りもあります）

練習 3: パイプラインの構築

以下の処理をひとつのパイプラインで書いてみてください。

1. 土曜日 (Sat) のデータを抽出。
2. チップ率を計算し、新しい列として追加。
3. そのチップ率が高い順（降順）に並べ替え。
4. 上位 5 件を表示。

第3回 記述統計 (1) 数値要約

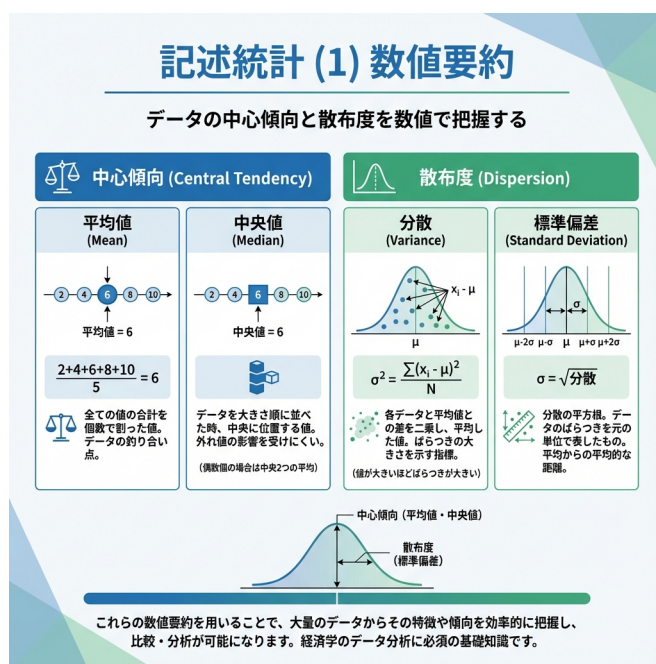


Figure3: 記述統計 (1) 数値要約

情報を「縮約」する

手元に1万人の顧客データがあるとします。上司に「客層はどうだい?」と聞かれたら、どう答えますか。1万人全員の年齢を読み上げるわけにはいきません。「だいたい30代です」と答えるはずです。

記述統計 (Descriptive Statistics) の本質は、ここにあります。膨大な生のデータを、たった1つの数字 (要約統計量) にまで圧縮する。多くの情報は捨てられますが、代わりに人

間が理解できる「特徴」が浮かび上がります。

本節では、この「情報の圧縮技術」を学びます。キーとなるのは次の2点です。

1. どこに集まっているか（中心的傾向）
2. どれくらい散らばっているか（散布度）

準備

前回に引き続き、pandas と tips データセット (Bryant & Smith, 1995) を使います。

```
import pandas as pd

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)
```

中心を探す：代表値

まずはデータの「顔」となる中心点を探します。ただ、「中心」の定義は一つではありません。

1. 全員の声を聞く：平均値 (Mean)

すべてのデータを足し合わせ、人数で割る。一番なじみ深い「中心」でしょう。

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

```
tips["total_bill"].mean()
```

19.78594262295082

この指標には「全員の値を公平に扱う」という民主的な良さがあります。反面、**極端な声（外れ値）に弱い**のが欠点です。たった一人の超富裕層がいるだけで、全員の平均所得は跳ね上がってしまいます。

2. 真ん中の人を見る：中央値 (Median)

データを一行に並べたとき、ちょうど真ん中にいる人の値です。

```
tips["total_bill"].median()
```

```
17.795
```

こちらは「順位」しか見ないので、極端な値の影響をほとんど受けません（頑健性）。

3. 多派を見る：最頻値 (Mode)

最も頻繁に現れる値です。数値データよりも、アンケートの回答（賛成/反対）などのカテゴリデータで威力を発揮します。

```
# 最頻値は value_counts() で確認できる
tips["day"].value_counts()
```

```
day
```

```
Sat      87
```

```
Sun      76
```

```
Thur     62
```

```
Fri      19
```

```
Name: count, dtype: int64
```

意思決定：平均か、中央値か

どっちを使うべきか。迷ったら「世界の形（分布）」を見ます。

分布の形状	選び方の指針
左右対称（テストの点数など）	どちらもほぼ同じ値になります。数学的に扱いやすい 平均値 を選びます。

歪んでいる（年収、貯蓄額など） 右に裾が長い場合、平均値は高すぎる値になります。実感を反映する**中央値**を選びます。

広がりを測る：散布度

データの特徴は「中心」だけでは語れません。「平均年収 500 万」と言っても、全員が 500 万の国と、0 円と 1000 万がいる国では全く別物です。この「散らばり具合」を数値にします。

最大と最小：範囲 (Range)

一番上と一番下の差です。単純ですが、全体像を掴むにはこれで十分なことも多いです。

```
# 最小値と最大値
tips["total_bill"].min(), tips["total_bill"].max()

# その差（範囲）
tips["total_bill"].max() - tips["total_bill"].min()
```

47.74

中心からの距離：分散 (Variance)

「平均からどれくらい離れているか」を全データについて計算し、平均したもの。直感的には「差の絶対値」を平均したくなりますよね。でも、統計学では数学的な都合から「差を二乗」して扱います。

Python (pandas) の注意点: `pandas.Series.var()` は、手元のデータを標本とみなした**不偏分散**（分母が $n - 1$ 、`ddof=1`）がデフォルトです。

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

```
tips["total_bill"].var()
```

79.25293861397826

単位を戻す：標準偏差 (Standard Deviation)

分散は計算過程で「二乗」しているため、単位が「ドルの二乗」のようになり、直感的に分かりにくい。そこで、平方根をとって元の単位に戻したのが**標準偏差**です。

$$s = \sqrt{s^2}$$

```
tips["total_bill"].std()
```

8.902411954856856

「平均値 ± 標準偏差」の範囲に、多くのデータ（正規分布なら約 68%）が含まれると考えるとイメージしやすいでしょう。

単位を消す：変動係数 (CV)

「象の体重のばらつき」と「アリの体重のばらつき」を比べたいとき、標準偏差 (kg や g) そのままでは比較できません。平均値に対する比率 (%) に直すことで、スケールの異なるデータでも比べられます。

$$CV = \frac{s}{\bar{x}} \times 100\%$$

```
# チップの額と、支払総額。どちらが「ばらついて」いるか？
```

```
cv_tip = tips["tip"].std() / tips["tip"].mean() * 100
```

```
cv_bill = tips["total_bill"].std() / tips["total_bill"].mean() * 100
```

```
print(f"チップの変動係数: {cv_tip:.2f}%")
```

```
print(f"支払総額の変動係数: {cv_bill:.2f}%")
```

チップの変動係数: 46.15%

支払総額の変動係数: 44.99%

全体像をスキャンする：四分位数

データを小さい順に並べて4等分した地点（25%, 50%, 75%）を**四分位数**と呼びます。
`describe()` を使えば、これらの主要な統計量を一括で確認できます。

```
tips["total_bill"].describe()
```

```
count    244.000000
mean      19.785943
std       8.902412
min       3.070000
25%      13.347500
50%      17.795000
75%      24.127500
max       50.810000
Name: total_bill, dtype: float64
```

- **Median** (50% 地点) と **Mean** (平均) のズレを見ることで、データの歪みを推測できます。
- **1st Qu** (25%) から **3rd Qu** (75%) の間には、データの中央半数が含まれます。

安定した広がり：四分位範囲 (IQR)

データの中央 50% が収まる幅（第3四分位数 - 第1四分位数）を**四分位範囲**と呼びます。標準偏差と違い、極端な値の影響を受けないため、外れ値を含むデータでのばらつき指標として優秀です。

```
tips["total_bill"].quantile(0.75) - tips["total_bill"].quantile(0.25)
```

```
10.779999999999998
```

ノイズか、シグナルか：外れ値の検出

統計分析において、極端に離れた値（外れ値）は厄介な存在です。入力ミス（ノイズ）なのか、特異な事象（シグナル）なのかを見極める必要があります。一般的に、箱ひげ図の Tukey の基準を用いて「外れ値候補」を機械的に検出します。

- 下限: 第 1 四分位数 - $1.5 \times \text{IQR}$
- 上限: 第 3 四分位数 + $1.5 \times \text{IQR}$

```
Q1 = tips["total_bill"].quantile(0.25)
Q3 = tips["total_bill"].quantile(0.75)
IQR_val = Q3 - Q1

# 境界線の計算
lower_bound = Q1 - 1.5 * IQR_val
upper_bound = Q3 + 1.5 * IQR_val

print("外れ値の下限:", lower_bound)
print("外れ値の上限:", upper_bound)

# 境界を超えたデータを抽出してみる
tips.loc[
    (tips["total_bill"] < lower_bound) | (tips["total_bill"] > upper_bound),
    ["total_bill", "tip", "day", "time"],
]
```

外れ値の下限: -2.8224999999999945

外れ値の上限: 40.297499999999999

	total_bill	tip	day	time
59	48.27	6.73	Sat	Dinner
102	44.30	2.50	Sat	Dinner
142	41.19	5.00	Thur	Lunch

	total_bill	tip	day	time
156	48.17	5.00	Sun	Dinner
170	50.81	10.00	Sat	Dinner
182	45.35	3.50	Sun	Dinner
184	40.55	3.00	Sun	Dinner
197	43.11	5.00	Thur	Lunch
212	48.33	9.00	Sat	Dinner

分布の歪みを可視化する

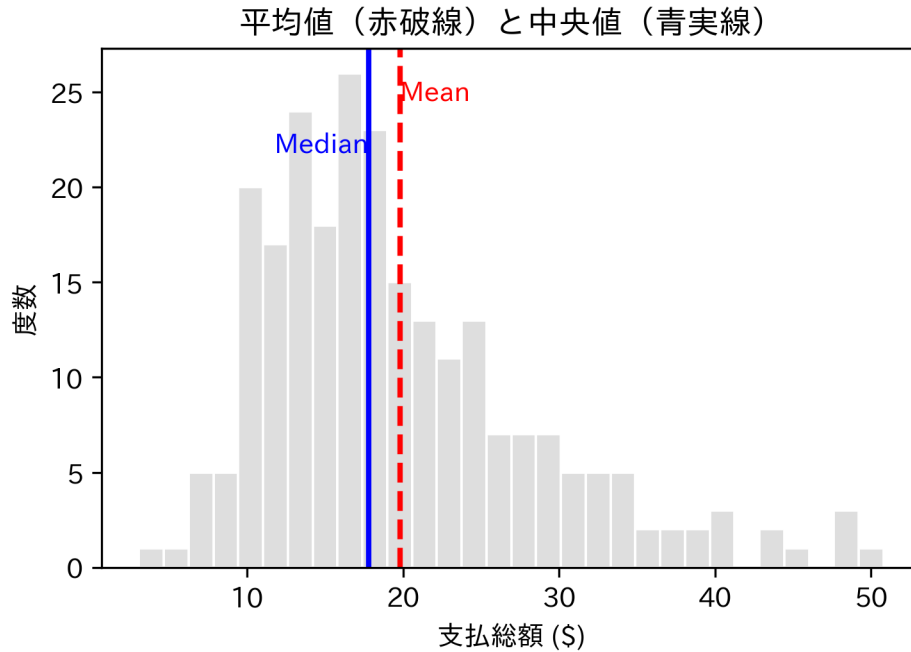
平均値と中央値がずれているとき、実際にグラフを描くとその理由がわかります。ヒストグラムに両者の位置を重ねてみましょう。

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

mean_val = tips["total_bill"].mean()
med_val = tips["total_bill"].median()

ax = sns.histplot(data=tips, x="total_bill", bins=30, color="lightgray", edgecolor="black")
ax.axvline(mean_val, color="red", linestyle="--", linewidth=2)
ax.axvline(med_val, color="blue", linestyle="-", linewidth=2)

y_top = ax.get_ylim()[1]
ax.text(mean_val, y_top * 0.9, "Mean", color="red", ha="left")
ax.text(med_val, y_top * 0.8, "Median", color="blue", ha="right")
ax.set(title="平均値（赤破線）と中央値（青実線）", xlabel="支払総額（$）", ylabel="度数")
plt.show()
```

グループごとに要約する

全体の平均だけでは見えない違いもあります。性別や喫煙の有無など、属性ごとに分けて要約することで、より深い洞察が得られます。

```
(
    tips
    .groupby("sex", as_index=False)
    .agg(
        avg_bill=("total_bill", "mean"),
        sd_bill=("total_bill", "std"),
        count=("total_bill", "size"), # サンプルサイズも必ず確認する
    )
)
```

	sex	avg_bill	sd_bill	count
0	Female	18.056897	8.009209	87
1	Male	20.744076	9.246469	157

課題

自分の手でデータを縮約してみましょう。

1. `tips` データを使用し、時間帯 (`time`) ごとのチップ (`tip`) の平均値を算出してください。
2. `describe()` を使用し、チップ (`tip`) の分布の要約統計量を確認してください。

練習問題

練習 1: 基本的な記述統計の算出

`tips` データの `size` (グループの人数) について、以下の 4 つの統計量を算出してください。

1. 平均値
2. 中央値
3. 標準偏差
4. 範囲 (最小値と最大値)

練習 2: グループ別の比較検証

喫煙者 (`smoker == "Yes"`) と非喫煙者 (`smoker == "No"`) の 2 群において、支払総額 (`total_bill`) の平均値と標準偏差を比較してください。

練習 3: 外れ値の影響を体感する

1. `tips` データに極端な外れ値 (例: `total_bill = 100`) を持つ行を追加した新しいデータを作成してください。
2. 元のデータと新しいデータで、平均値と中央値がそれぞれどれくらい変化したかを確認してください。どちらが「頑健」でしたか？

※本節で用いた記述統計の指標一覧は、練習問題の振り返りとして自身のノートにまとめ

ておくことを推奨します。

第4回 記述統計 (2) データの可視化

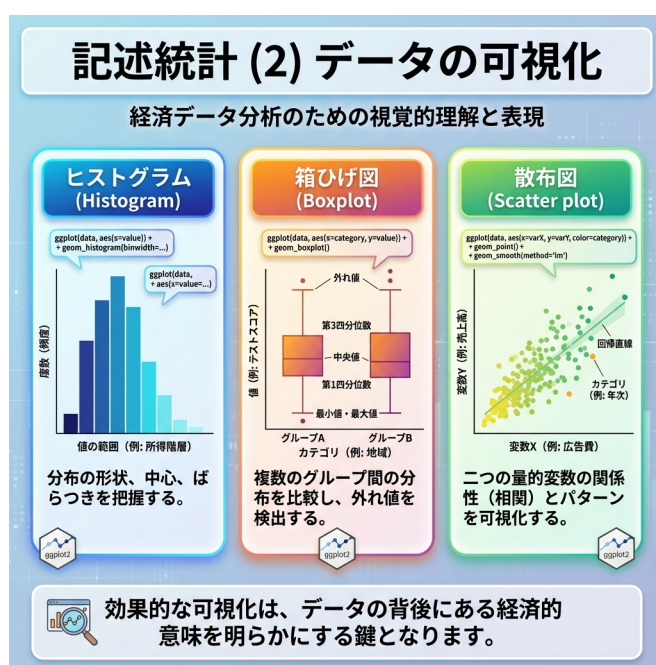


Figure4: データの可視化

要約の限界と「アンスコムの警鐘」

前節では、膨大なデータを「平均値」や「分散」などの数値に縮約し、全体の特徴を掴みました。しかし、縮約には副作用があります。「形の消失」です。

Anscombe (1973) の有名な例、アンスコムのカルテットを見てみましょう。一見、平均

値も分散も相関係数もしっかり一致している4つのデータセットです。

```
import pandas as pd
import seaborn as sns

# seaborn に用意されている anscombe (アンスコムのカルテット)
anscombe = sns.load_dataset("anscombe") # columns: dataset, x, y

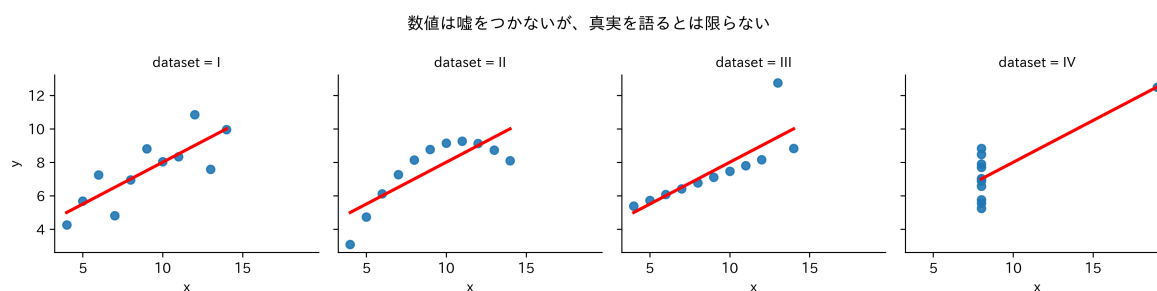
stats = (
    anscombe
    .groupby("dataset")
    .agg(
        mean_x=("x", "mean"),
        mean_y=("y", "mean"),
        sd_x=("x", "std"),
        sd_y=("y", "std"),
    )
)
stats["cor"] = anscombe.groupby("dataset").apply(lambda g: g["x"].corr(g["y"]))
stats
```

	mean_x	mean_y	sd_x	sd_y	cor
dataset					
I	9.0	7.500909	3.316625	2.031568	0.816421
II	9.0	7.500909	3.316625	2.031657	0.816237
III	9.0	7.500000	3.316625	2.030424	0.816287
IV	9.0	7.500909	3.316625	2.030579	0.816521

数値だけなら「これらは同じ性質のデータだ」と結論づけてしまうでしょう。でも、グラフにして可視化した瞬間、景色は一変します。[対象注意: 日本語フォント設定は環境依存あり]

```
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

g = sns.lmplot(
    data=anscombe,
    x="x",
    y="y",
    col="dataset",
    ci=None,
    height=3,
    scatter_kws={"s": 35, "alpha": 0.9},
    line_kws={"color": "red"},
)
g.set_axis_labels("x", "y")
g.fig.suptitle("数値は嘘をつかないが、真実を語るとは限らない")
g.fig.tight_layout()
plt.show()
```



きれいな直線、曲線、外れ値の影響…。数値要約で捨て去られた情報が、可視化ではっきりと浮かび上がりました。データ分析において、可視化は分析の「後」の飾りではありません。分析の「最初」に行うべき、欠かせない診断です。

準備

本節でも `tips` データセット (Bryant & Smith, 1995) を使います。

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
import pandas as pd

tips = pd.read_csv(url)
```

キャンバスに層を重ねる：seaborn / matplotlib の考え方

Python の可視化では、**seaborn**（統計可視化）と **matplotlib**（土台）が定番です。Excel みたいに「棒グラフを作るボタン」があるわけではありません。

「データ」という土台に、「座標」を定義し、「点や棒（Geom）」を乗せ、「説明（Labels）」を加える。

この工程をコードで書きます。

```
import seaborn as sns
import matplotlib.pyplot as plt

ax = sns.scatterplot(data=tips, x="total_bill", y="tip") # キャンバスと座標+点
plt.show()
```

1. 分布の形状を見る

まずは、データが「どんな形」をしているか確認します。

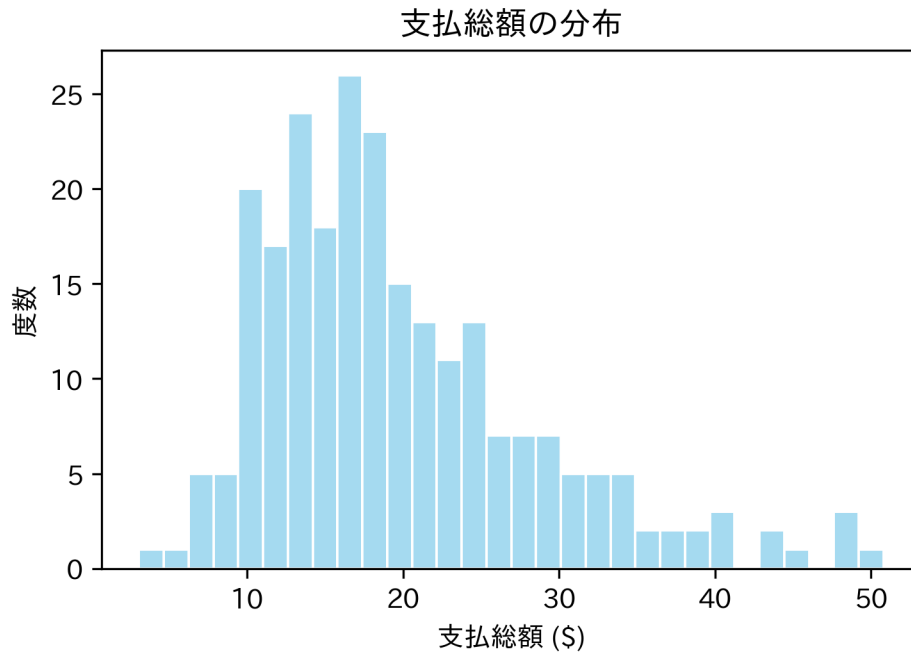
山の形を知る：ヒストグラム

連続した数値データ（金額など）が、どこに集まっているかを見に行きます。山は一つか、二つか？ 左右対称か、歪んでいるか？

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
```



```
ax = sns.histplot(data=tips, x="total_bill", bins=30, color="skyblue", edgecolor="white")
ax.set(title="支払総額の分布", xlabel="支払総額 ($)", ylabel="度数")
plt.show()
```

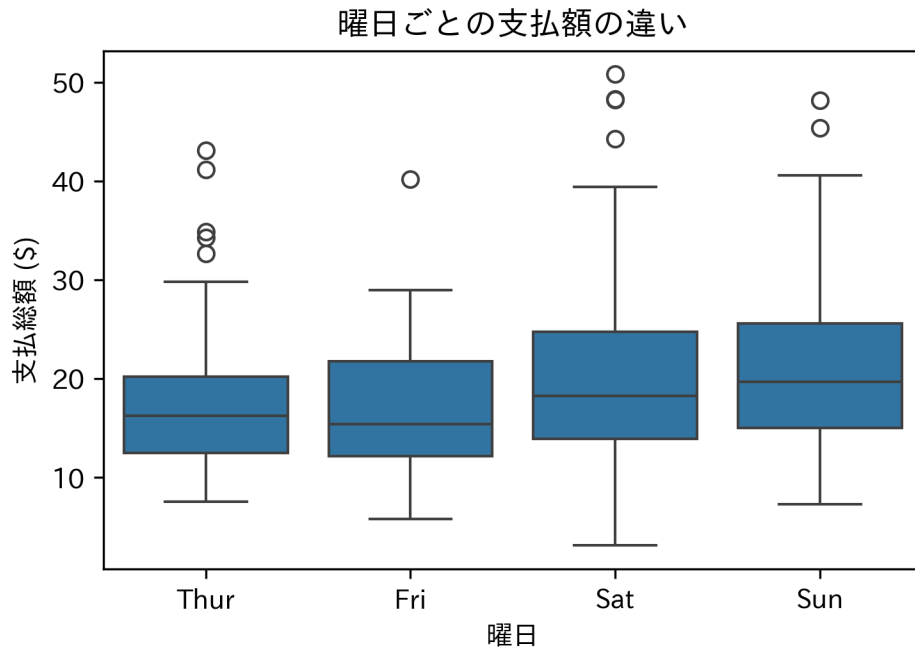


構造を要約する：箱ひげ図

ヒストグラムは詳細ですが、複数のグループ比較には不向きです。重なって見づらいから。そこで**箱ひげ図**の出番です。分布の情報を「箱」と「ひげ」に要約します。

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

order = ["Thur", "Fri", "Sat", "Sun"]
ax = sns.boxplot(data=tips, x="day", y="total_bill", order=order)
ax.set(title="曜日ごとの支払額の違い", xlabel="曜日", ylabel="支払総額 ($)")
plt.show()
```



Tukey の箱ひげ図の定義:

- **箱**: データの中央 50% (第 1 四分位数～第 3 四分位数)。この中に「普通」のデータが入ります。
- **太線**: 中央値 (Median)。
- **点**: 箱から 1.5 倍の四分位範囲以上離れた値。統計的な「外れ値候補」として扱います。

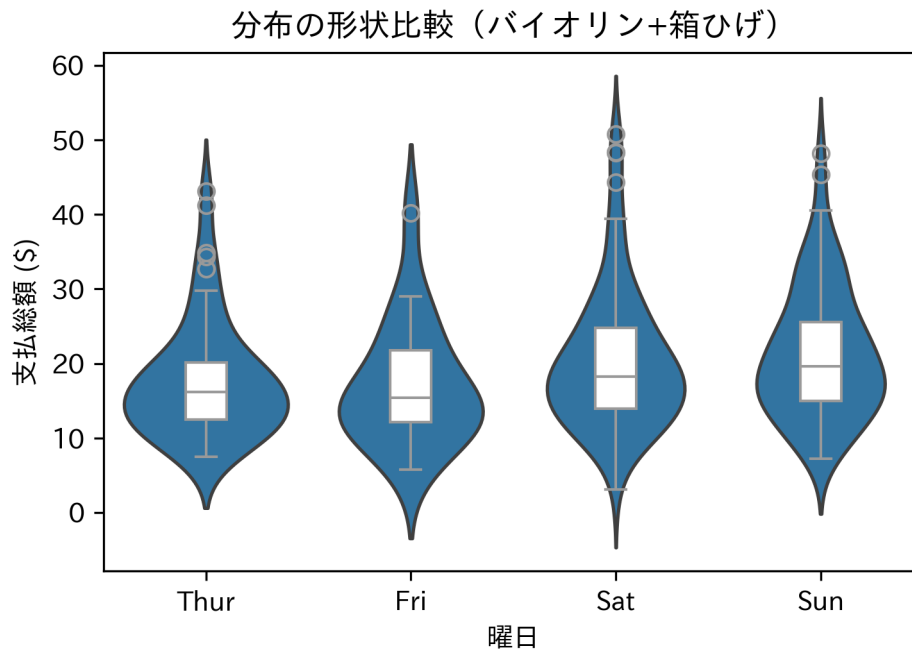
分布の機微を見る：バイオリンプロット

箱ひげ図だと「山が 2 つある」などの情報は消えてしまいます。分布の形状 (密度) そのものを表示したい場合は、バイオリン図を使います。

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

order = ["Thur", "Fri", "Sat", "Sun"]
ax = sns.violinplot(data=tips, x="day", y="total_bill", order=order, inner=None)
sns.boxplot(data=tips, x="day", y="total_bill", order=order, width=0.2, color="wh")
```

```
ax.set(title="分布の形状比較（バイオリン+箱ひげ）", xlabel="曜日", ylabel="支払総額 ($)")
plt.show()
```



2. 関係性を探る

次は、2 つの変数はどう関わっているかを探ります。

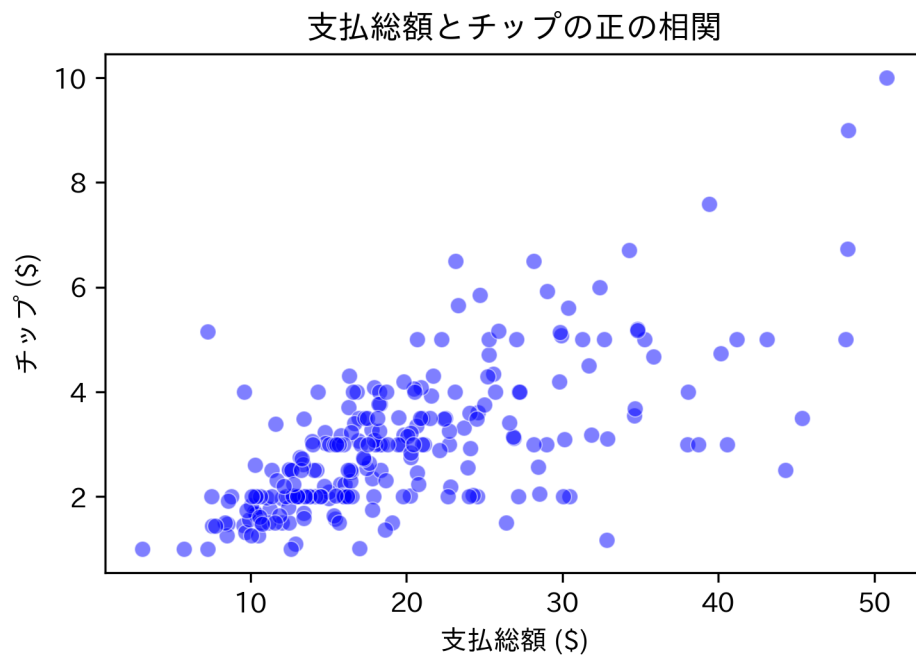
2 つの量の相関：散布図

「たくさん食べた人は、たくさんチップを払うのか？」2 つの連続変数（横軸と縦軸）の関係をプロットします。

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

ax = sns.scatterplot(data=tips, x="total_bill", y="tip", color="blue", alpha=0.5)
ax.set(title="支払総額とチップの正の相関", xlabel="支払総額 ($)", ylabel="チップ ($)")
```

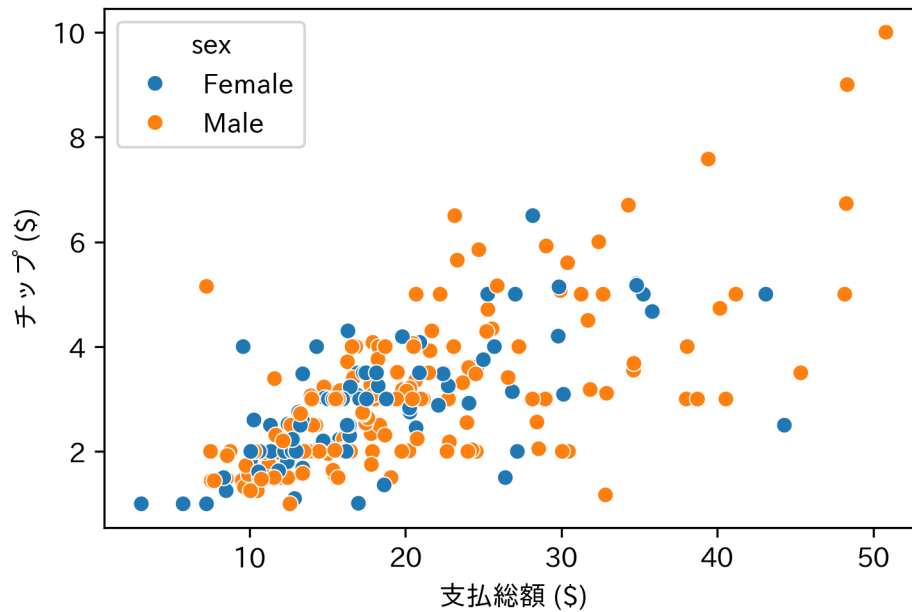
```
plt.show()
```



ここに第3の次元（情報）を加えるのも簡単です。例えば、性別によって色を変えます（`color = sex`）。

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

ax = sns.scatterplot(data=tips, x="total_bill", y="tip", hue="sex")
ax.set(xlabel="支払総額 ($)", ylabel="チップ ($)")
plt.show()
```



状況ごとの違い：ファセット分割

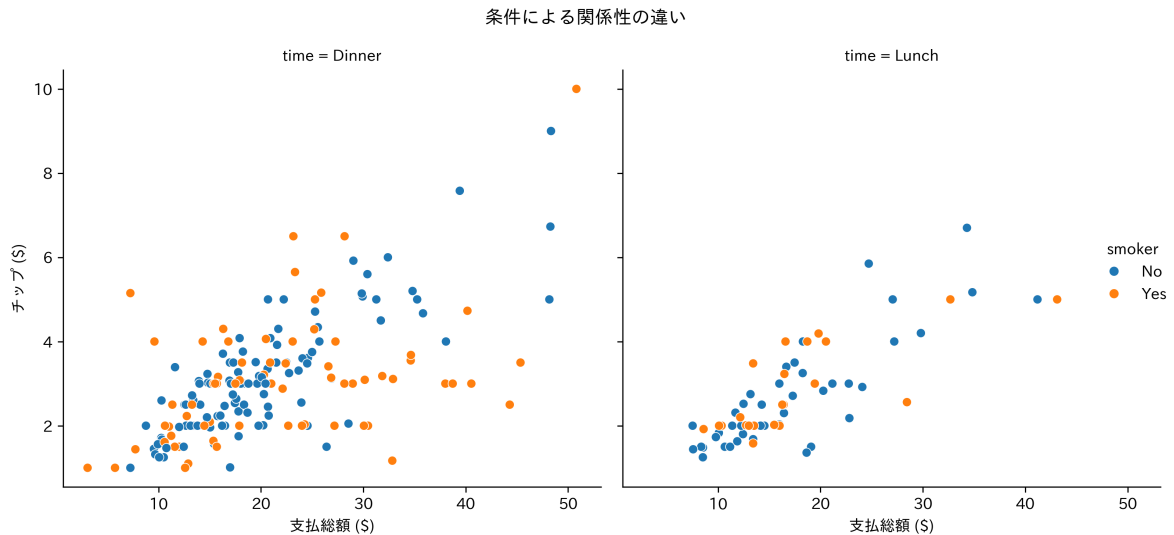
色分け (color) は便利ですが、情報が増えすぎると「スパゲッティ」のように絡まって読めなくなる。そんなときは `facet_wrap()` でグラフ自体を分割してしまいましょう。

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

g = sns.relplot(
    data=tips,
    x="total_bill",
    y="tip",
    hue="smoker",
    col="time",
    kind="scatter",
)

g.set_axis_labels("支払総額 ($)", "チップ ($)")
g.fig.suptitle("条件による関係性の違い")
```

```
g.fig.tight_layout()
plt.show()
```



グラフを「語らせる」ための調整

デフォルトのままでは、グラフは「下書き」です。誰かに伝えるためには、意図が伝わるよう調整します。

統計情報の重ね書き

「傾向」を見せたいなら、生のデータ点だけでなく、平均値や回帰直線を重ねるのが効果的です。

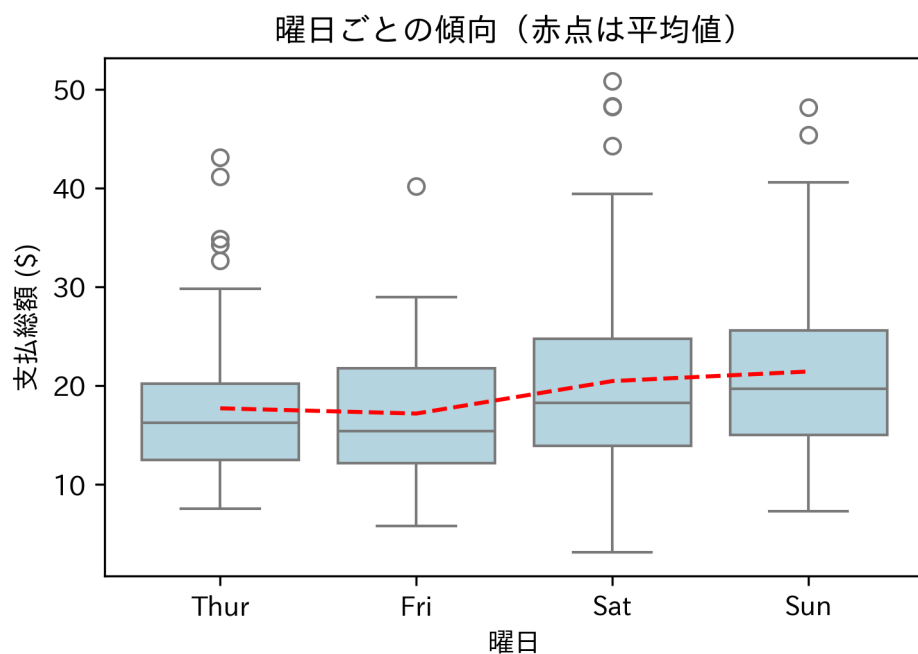
```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

order = ["Thur", "Fri", "Sat", "Sun"]
ax = sns.boxplot(data=tips, x="day", y="total_bill", order=order, color="lightblue")

means = tips.groupby("day")["total_bill"].mean().reindex(order)
```

```
xs = range(len(order))
ax.scatter(xs, means.values, color="red", s=60)
ax.plot(xs, means.values, color="red", linestyle="--")

ax.set(title="曜日ごとの傾向（赤点は平均値）", xlabel="曜日", ylabel="支払総額（$）")
plt.show()
```



ユニバーサルデザインへの配慮

あなたのグラフを見る人の中には、特定の色が見えにくい人（色覚多様性）がいるかもしれません。「赤と緑」の色分けだけで情報を伝えてはダメです。形状（`shape`）や線種（`linetype`）の違いを併用するか、誰もが識別しやすいカラーパレット（Viridis など）の使用をおすすめします。

課題

自分の手で可視化を行い、データからの発見を言語化してください。

1. `tip` (チップ) の分布をヒストグラムで確認してください。何か気づく特徴はありますか? (キリの良い数字が多い、など)
2. `total_bill` と `tip` の散布図を描き、`time` (ランチ/ディナー) で色分けして、時間帯による傾向の違いを議論してください。

練習問題

練習 1: ヒストグラムの最適化

1. `tip` のヒストグラムを作成してください。
2. ビンの数 (`bins`) を 10, 30, 50 と変えてみてください。分布の印象がどう変わるか観察してください。

練習 2: 多変量の可視化

1. 性別 (`sex`) ごとの `tip` の分布を箱ひげ図で比較してください。女性の方がばらつきが大きいですか?
2. 喫煙者 (`smoker`) ごとの `total_bill` の分布を、`facet_wrap()` を使って左右に並べて比較してください。

練習 3: 散布図への情報付加

1. `total_bill` と `tip` の散布図を作成してください。
2. 点の大きさを `size` (人数) にマッピングしてください (`aes(size = size)`)。大人数のグループは右上に集まっていますか?
3. 全体の傾向を示す回帰直線を `geom_smooth()` で追加してください。

第 5 回確率論の基礎

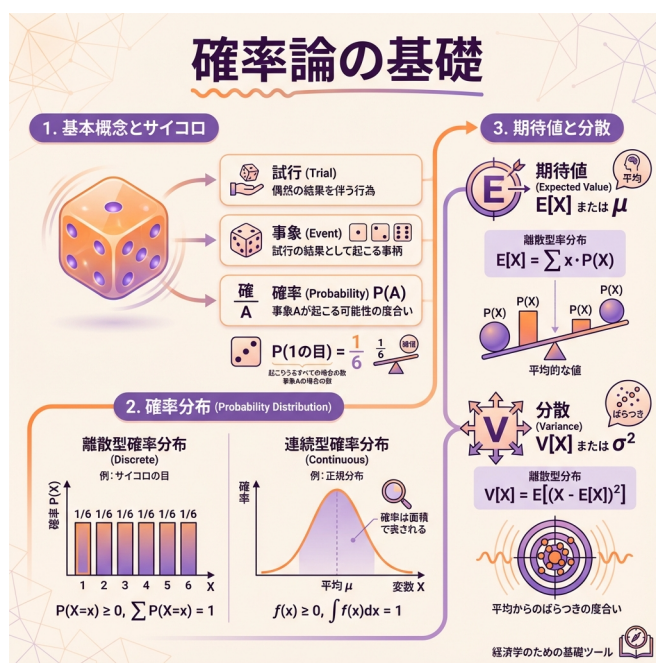


Figure5: 確率論の基礎

不確実性を飼いならす言語

データ分析のゴールは、手元のデータ（標本）から背後の世界（母集団）を推測すること。でも、そこには必ず「ズレ」があります。今日のデータと明日のデータは、違うから。

この「ズレ」や「不確実性」を、「なんとなく」ではなく数学的に扱うための言語。それが確率論です。統計学を学ぶ私たちにとって、避けては通れない共通言語です。

可能性の地図：確率分布

「次に何が起こるか」は誰にもわかりません。でも、「何がどれくらいの可能性で起こるか」の全体像なら描けます。

- **確率変数 (X):** コインの表裏やサイコロの目のように、結果が偶然で決まる変数。
- **確率分布:** 変数がとりうる値と、その確率の対応関係。いわば「可能性の地図」です。

デジタルとアナログ：離散と連続

確率変数には、大きく 2 つのタイプがあります。

タイプ	イメージ	具体例	特徴
離散型 (Discrete)	デジタル (飛び飛び)	サイコロの目、人数	1 つ 1 つの値に確率がある ($P(X = x)$)
連続型 (Continuous)	アナログ (連続)	身長、時間、気温	一点狙いの確率はゼロ。面積で確率を考える

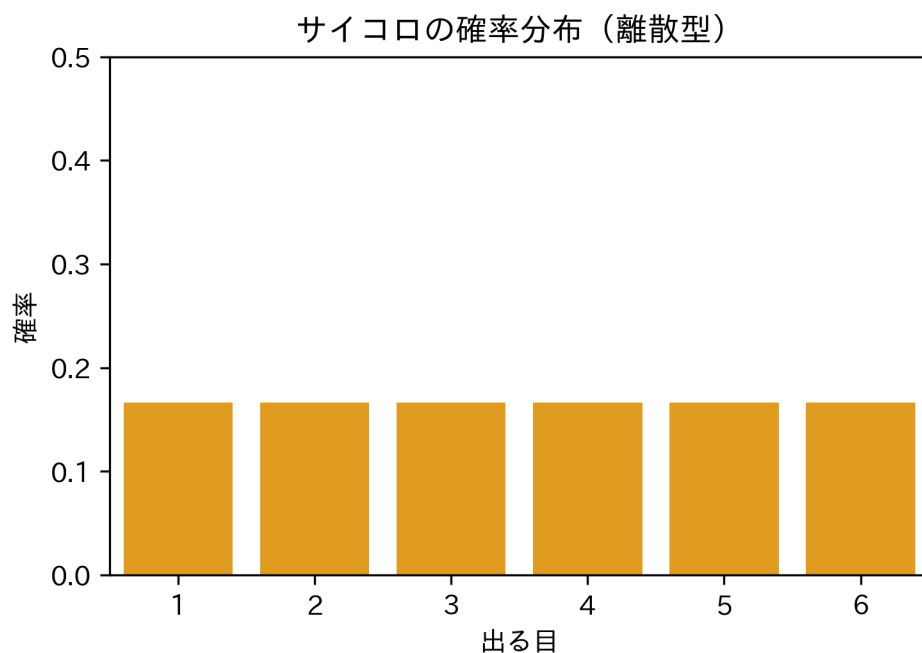
離散型の実験：サイコロの地図

一番シンプルな離散分布、「公正なサイコロ」を考えます。1 から 6 までの目が、すべて均等に $1/6$ の確率で出ます。

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
```

```
x = np.arange(1, 7)
prob = np.repeat(1 / 6, 6) # 全て 1/6

df = pd.DataFrame({"x": x.astype(str), "prob": prob})
ax = sns.barplot(data=df, x="x", y="prob", color="orange")
ax.set(ylim=(0, 0.5), title="サイコロの確率分布（離散型）", xlabel="出る目", ylabel="確率")
plt.show()
```



分布の特徴を捉える

地図全体を眺めるのは大変です。特徴を一言で表す「要約」がほしい。記述統計で学んだ「平均」と「分散」。こいつらが、確率論の世界では**期待値**と**分散**として再登場します。

1. 期待値 (Expected Value): 「賭け」の相場

もしサイコロ博打を何千回も繰り返したら、平均で何点とれるか。それが期待値 $E[X]$ 。分布の「重心」です。

$$E[X] = \sum_i x_i P(X = x_i)$$

サイコロなら：

$$1 \times \frac{1}{6} + \dots + 6 \times \frac{1}{6} = 3.5$$

「3.5」という目は実際には出ません。でも、長く続ければこの値に落ち着く。これがこのゲームの「平均的な相場」です。

2. 分散 (Variance): リスクの大きさ

期待値（重心）から、どれくらい離れた値が出やすいか。このブレこそ、リスク。統計学では分散 $V[X]$ と呼びます。

$$V[X] = E[(X - E[X])^2]$$

```
E_X = (x * prob).sum()
V_X = ((x - E_X) ** 2 * prob).sum()

print("期待値:", E_X)
print("分散:", V_X)
print("標準偏差:", np.sqrt(V_X))
```

期待値: 3.5

分散: 2.9166666666666665

標準偏差: 1.707825127659933

線形性：計算を楽にする魔法

期待値と分散には、計算を劇的に楽にする性質（魔法）があります。例えば、「サイコロの目を 10 倍して 3 を足したスコア」の平均を知りたいとき、いちいちシミュレーションしなくていい。

- 期待値: $E[10X + 3] = 10E[X] + 3 = 10(3.5) + 3 = 38$
- 分散: $V[10X + 3] = 100V[X]$ (定数の足し算は無視。掛け算は二乗で効く)

連続の世界へ

身長や時間など、連続的な数値の場合、「ちょうど 170.000...cm である確率」は実質ゼロです。そこで、**確率密度関数 (PDF)** を使い、「点」ではなく「面積」で確率を考えます。

確率密度関数とは

- $f(x)$ というカーブの下側の面積が、確率になります。
- 全区間の面積 (全確率) は、必ず 1。

実験：一様分布シミュレーション

0 から 1 までの乱数を発生させる「一様分布」で、理論と現実を比べてみます。理論上の期待値は 0.5、分散は $1/12 \approx 0.083$ です。

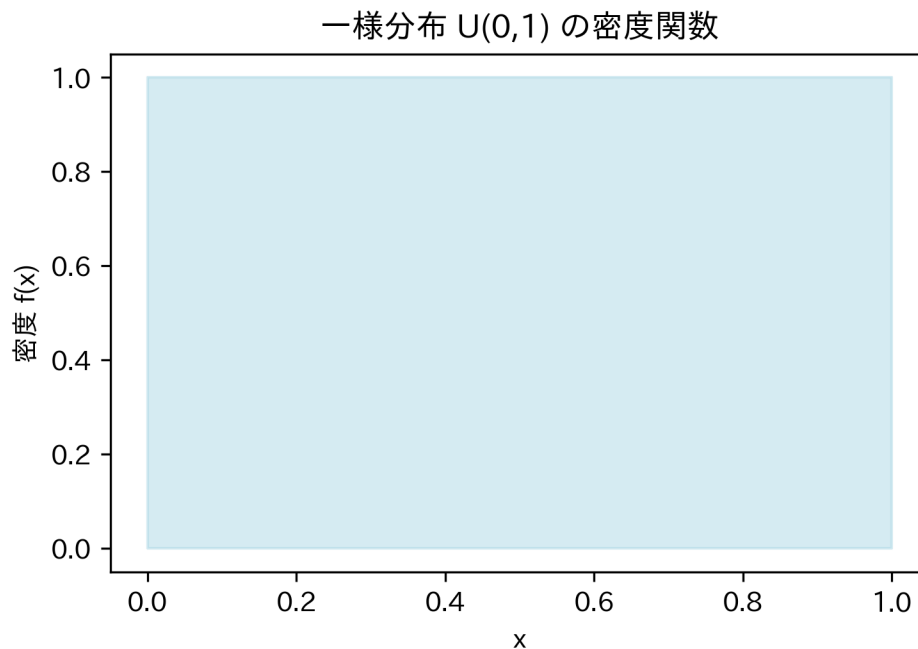
```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import uniform

# 1. 密度関数の可視化 (理論)
xs = np.linspace(0, 1, 200)
ys = uniform.pdf(xs, loc=0, scale=1)
plt.fill_between(xs, ys, color="lightblue", alpha=0.5)
plt.title("一様分布 U(0,1) の密度関数")
plt.ylabel("密度 f(x)")
plt.xlabel("x")
plt.show()

# 2. シミュレーション (現実)
```

```
rng = np.random.default_rng(123)
samples = rng.random(10000)

print("--- シミュレーション結果 ---")
print("サンプル平均:", samples.mean(), " (理論値: 0.5)")
print("サンプル分散:", samples.var(ddof=1), " (理論値: 1/12 ≈ 0.0833...)")
```



--- シミュレーション結果 ---

サンプル平均: 0.49440755879181897 (理論値: 0.5)

サンプル分散: 0.08146389039123135 (理論値: $1/12 \approx 0.0833\dots$)

回数を重ねるほど、現実は理論（期待値）に近づいていきます。**大数の法則**です。

同時確率と独立性

独立であるとはどういうことか

2つの事象 A と B が「独立」。これは、お互いに影響し合わないこと。数学的な定義はシンプルです。

$$P(A \cap B) = P(A) \times P(B)$$

「A かつ B が起こる確率」が、単純な掛け算で計算できるなら、それは独立。逆に、掛け算で合わないなら、裏で何かがつながっています（相関や因果）。

課題/練習問題

自力で計算し、Python で確かめてみてください。

課題 1: 歪んだコインのギャンブル

表が出る確率が 0.1、裏が 0.9 のコインがあります。表なら 100 円、裏なら 0 円もらえると、このギャンブルの期待値（参加料の適正価格）はいくらですか？

課題 2: 独立性の検証

「雨が降る確率」が 30%、「傘を持つ確率」が 50% だとします。もし「雨が降り、かつ傘を持っている確率」が 15% なら、雨と傘は独立と言えますか？（ヒント： 0.3×0.5 と比べてください）

※本節の公式（期待値・分散の定義、線形性）は、次回の「推測統計」でフル活用します。手計算できるようになっておいてください。

第 6 回主要な確率分布

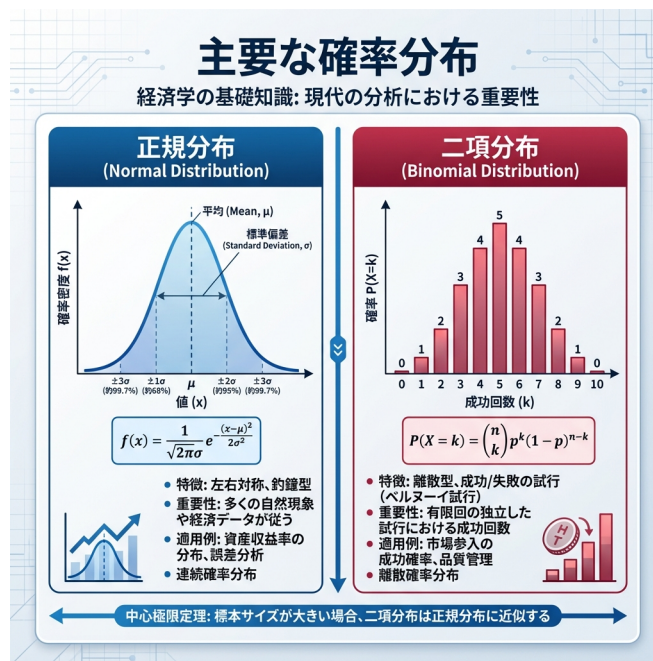


Figure6: 主要な確率分布

世界を支配する「釣り鐘」

身長、テストの点数、工場の部品サイズ、株価の変動…。まったく異なるこれらすべての現象に、どこにでも現れる不思議な曲線があります。**正規分布 (Normal Distribution)**。またの名を、**ガウス分布**。

なぜ世界は、ことあるごとに正規分布したがるのか？ その秘密は、統計学で最強の定理 (中心極限定理) にあります。まずはこの「王様」の振る舞いを知ることから始めましょう。

本節では、統計解析の主役である正規分布と、その親戚たち（二項分布、t 分布など）を、Python を使って自在に操れるようになります。

1. 統計学の王様：正規分布

正規分布は、平均 μ と分散 σ^2 。たった 2 つのパラメータで完全に決まる、左右対称の美しい釣鐘型をしています。

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

数式は複雑に見えますが、Python なら一瞬で描けます。

4 つの基本操作

Python では **SciPy** (`scipy.stats`) を使うのが定番です。確率分布は共通のインターフェースを持ちます。

- **PDF/PMF**: 密度（連続）/確率（離散）。グラフの高さ。(norm.pdf, binom.pmf)
- **CDF**: 累積確率。ある地点より左側の面積。(norm.cdf)
- **PPF**: 分位点。確率から値を逆算。(norm.ppf)
- **RVS**: 乱数生成。シミュレーション用。(norm.rvs)

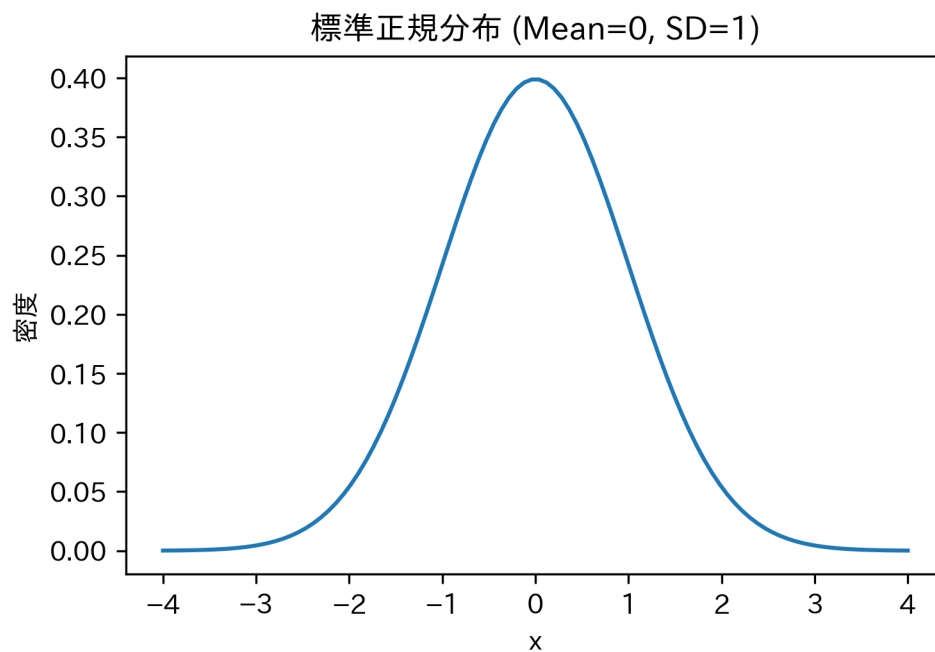
正規分布を描いてみる

まずは標準正規分布（平均 0、分散 1）を見てみましょう。

```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import norm

x = np.linspace(-4, 4, 100)
y = norm.pdf(x, loc=0, scale=1)
```

```
plt.plot(x, y)
plt.title("標準正規分布 (Mean=0, SD=1)")
plt.xlabel("x")
plt.ylabel("密度")
plt.show()
```



パラメータを変えてみる

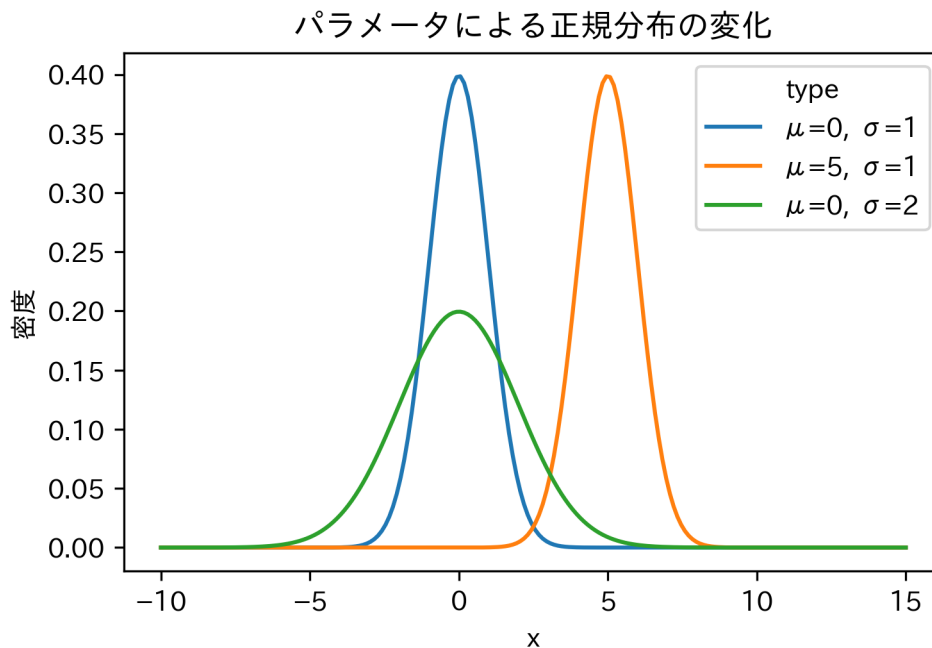
平均がズレると山が移動し、標準偏差が大きくなると山が平らになります。

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import norm

x = np.linspace(-10, 15, 200)
```

```
df_compare = pd.concat([
    pd.DataFrame({"x": x, "y": norm.pdf(x, loc=0, scale=1), "type": "μ=0, σ=1"},
    pd.DataFrame({"x": x, "y": norm.pdf(x, loc=5, scale=1), "type": "μ=5, σ=1"},
    pd.DataFrame({"x": x, "y": norm.pdf(x, loc=0, scale=2), "type": "μ=0, σ=2"}
    ],
    ignore_index=True,
)

ax = sns.lineplot(data=df_compare, x="x", y="y", hue="type")
ax.set(title="パラメータによる正規分布の変化", xlabel="x", ylabel="密度")
plt.show()
```



現実の問いに答える：確率計算

「平均 50、標準偏差 10 のテストで、60 点以上をとる確率は？」この問いに答えるのが p (累積確率) の関数です。

`pnorm(60)` は「 $-\infty$ から 60 までの面積」を返します。「60 点以上」を知りたいなら、全体 (100% = 1) から「60 点以下」の確率を引けばいい。

```
from scipy.stats import norm

# P(X >= 60) = 1 - P(X <= 60)
1 - norm.cdf(60, loc=50, scale=10)
```

0.15865525393145707

約 15.9%。クラスの 16% は、偏差値 60 以上ということ。

68-95-99.7 ルール

正規分布には、覚えておくべき相場観があります。「データの大半は、平均から標準偏差の 3 倍以内に収まる」というルールです。

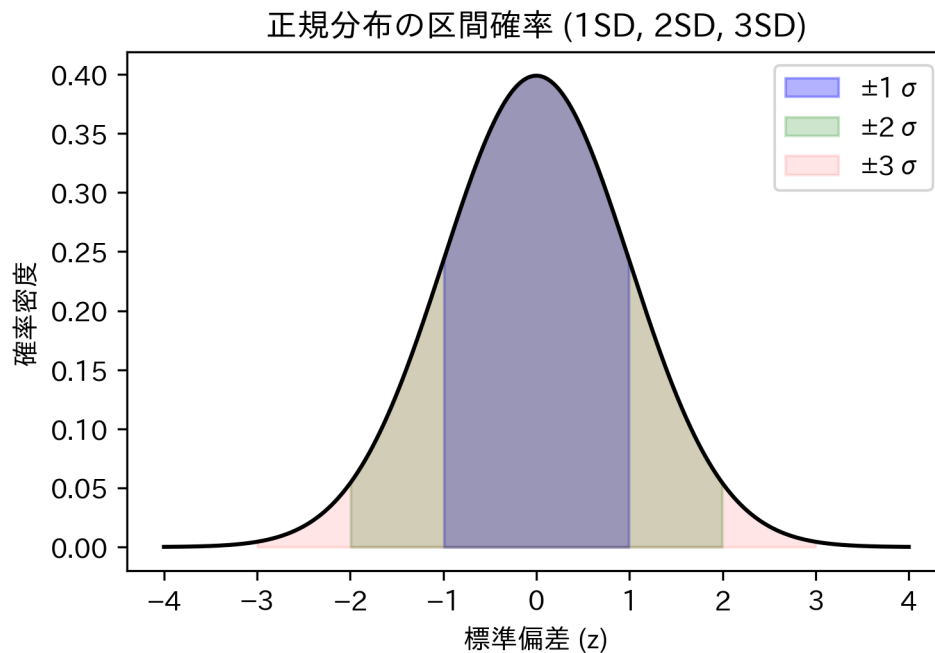
- $\pm 1\sigma$: 約 68% (偏差値 40~60)
- $\pm 2\sigma$: 約 95% (偏差値 30~70)
- $\pm 3\sigma$: 約 99.7% (ほぼすべて)

```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import norm

x = np.linspace(-4, 4, 400)
y = norm.pdf(x)

plt.plot(x, y, color="black")
plt.fill_between(x, 0, y, where=(x >= -1) & (x <= 1), color="blue", alpha=0.3, label="±1σ")
plt.fill_between(x, 0, y, where=(x >= -2) & (x <= 2), color="green", alpha=0.2, label="±2σ")
plt.fill_between(x, 0, y, where=(x >= -3) & (x <= 3), color="red", alpha=0.1, label="±3σ")
plt.title("正規分布の区間確率 (1SD, 2SD, 3SD)")
plt.xlabel("標準偏差 (z)")
```

```
plt.ylabel("確率密度")
plt.legend()
plt.show()
```



2. コイン投げの積み重ね：二項分布

次に、離散型の代表、**二項分布**。「成功か失敗か」のギャンブル（ベルヌーイ試行）を n 回繰り返したとき、何回勝てるか？ を表す分布です。

二項分布の 3 条件

単なる繰り返しではなく、3つの条件が必要です。1. 回数 n が決まっている。2. 毎回のリセット（独立性）。前の結果に引きずられない。3. 勝率 p が変わらない。

シミュレーション：運命の分かれ道

コイン投げ ($p = 0.5$) を 10 回行う勝負を考えます。「ちょうど 5 回勝つ確率」はどれくらいか。

```
from scipy.stats import binom
```

```
# pmf で確率（点）を計算
```

```
binom.pmf(5, n=10, p=0.5)
```

0.24609375000000002

約 24.6%。意外と低いですか？ すべてのパターンの確率を並べてみます。

```
import numpy as np
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
import japanize_matplotlib # noqa: F401
```

```
from scipy.stats import binom
```

```
x = np.arange(0, 11)
```

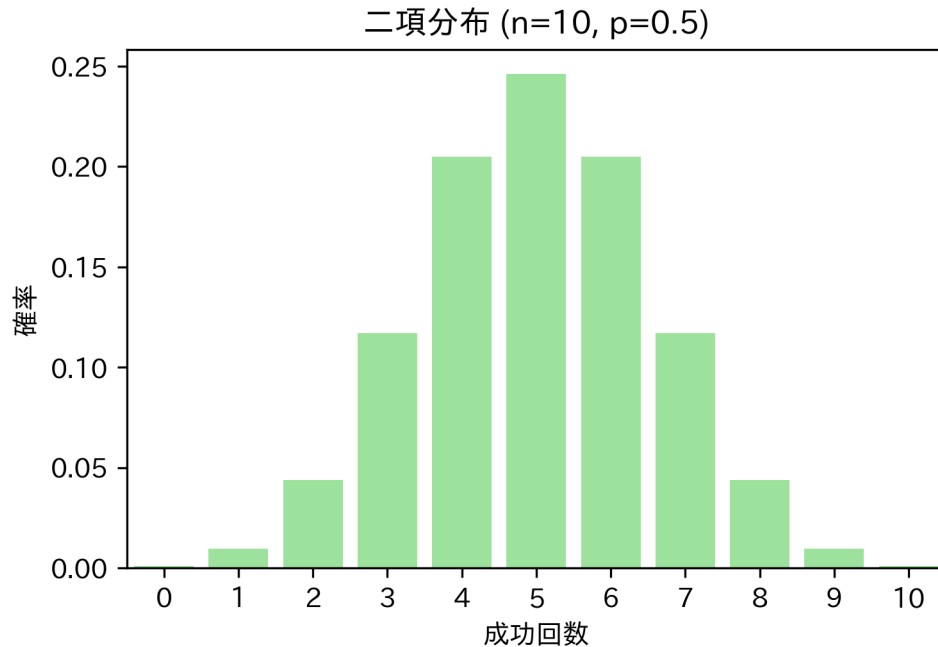
```
prob = binom.pmf(x, n=10, p=0.5)
```

```
df_binom = pd.DataFrame({"x": x.astype(str), "prob": prob})
```

```
ax = sns.barplot(data=df_binom, x="x", y="prob", color="lightgreen")
```

```
ax.set(title="二項分布 (n=10, p=0.5)", xlabel="成功回数", ylabel="確率")
```

```
plt.show()
```



回数を増やすと、奇跡が起きる

ここからが統計学のハイライト。このカクカクした二項分布、試行回数 n をどんどん増やしていくと、どうなるか。

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import binom

# 試行回数 n を変化させた場合の分布形状（表示順を指定）
n_values = [10, 30, 100]
p = 0.3

rows = []
for n in n_values:
    x = np.arange(0, n + 1)
```



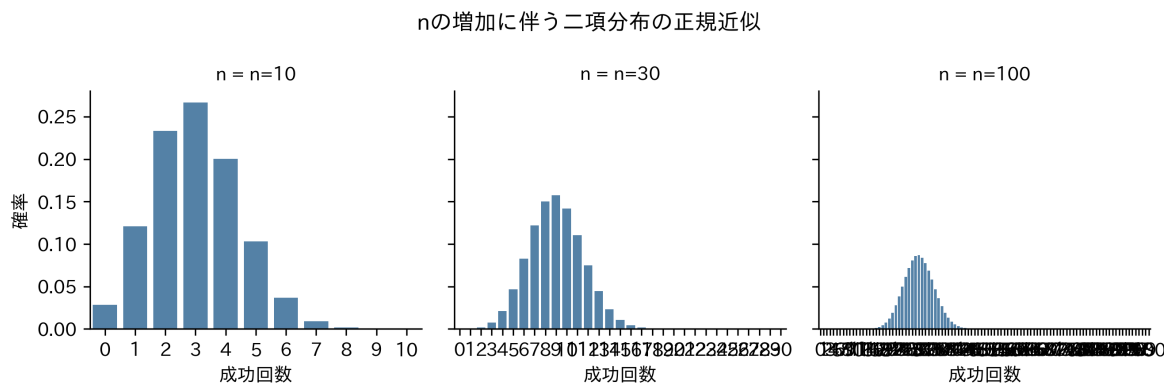
```

prob = binom.pmf(x, n=n, p=p)
rows.append(pd.DataFrame({"x": x, "prob": prob, "n": f"n={n}"}))

df_binom_approx = pd.concat(rows, ignore_index=True)

g = sns.FacetGrid(df_binom_approx, col="n", col_order=[f"n={n}" for n in n_values], sharex=True)
g.map_dataframe(sns.barplot, x="x", y="prob", color="steelblue")
g.set_axis_labels("成功回数", "確率")
g.fig.suptitle("n の増加に伴う二項分布の正規近似")
g.fig.tight_layout()
plt.show()

```



n が増えるにつれて、分布の左右対称性が増し、きれいな釣鐘型に近づいているのがわかります。「離散的なコイン投げ」が、積み重なることで「連続的な正規分布」に化けました。

これを**正規近似**（ド・モアブル＝ラプラスの定理）と呼び、後の**中心極限定理**へとつながる重要な性質です。

3. 正規分布の子供たち（派生分布）

最後に、推測統計の現場で頻繁に使われる 3 つの分布を紹介します。これらはすべて、正規分布を親として生まれた「正規分布ファミリー」。今は定義を丸暗記する必要はありません。「何のために生まれたか」という役割だけ、押さえておきます。

カイ二乗分布：ブレの分析官

- 出自: 標準正規分布の2乗和。
- 役割: 「分散（ばらつき）」を分析すること。
- 特徴: 2乗しているのでマイナスにはなりません。適合度検定などで活躍します。

t 分布：現場の代役

- 出自: 正規分布 $\div \sqrt{\chi^2/k}$ 。
- 役割: データ数（標本）が少ないときに、正規分布の代役を務める。
- 特徴: 正規分布より裾が厚く、外れ値のリスクを織り込んでいます。「母分散がわからない」という現実的な状況で、平均値を検定するための分布です。

F 分布：比較のスペシャリスト

- 出自: 2つのカイ二乗分布の比。
- 役割: 2つの「分散」を比べること。
- 特徴: 分散分析（ANOVA）で、グループ間の差が誤差より大きいかを判定する際に使われます。

課題

自らの手で確率を計算し、モデルの感覚を掴んでください。

1. **正規分布の計算:** 日本人成人男性の身長が平均 171cm、標準偏差 6cm の正規分布に従うと仮定します。身長が 180cm 以上の人は、上位何パーセントに含まれますか？
2. **二項分布の計算:** サイコロを 10 回振ったとき、1 の目が「ちょうど 2 回」出る確率は？（ヒント： $n = 10, p = 1/6$ ）

練習問題

練習 1: テストの点数分布

平均 50 点、標準偏差 10 点の正規分布に従うテストについて：

1. 40 点以下となる確率は？
2. 40 点から 60 点の間に入る確率は？
3. 上位 10% に入るには何点以上必要ですか？ (`qnorm` を使って逆算してください)

練習 2: 不良品の発生確率

不良品率が 10% ($p = 0.1$) の製造ラインで製品を 20 個 ($n = 20$) 作る場合：1. 不良品が 1 個も出ない確率 2. 不良品が 3 個以上出る確率 (`sum(dbinom(3:20, ...))`)

練習 3: 乱数シミュレーション

標準正規分布から乱数を 100 個生成しヒストグラムを描画してください。その後、サンプルサイズを 1,000 個、10,000 個と増やしたとき、ヒストグラムの形状がどのように理論分布（釣鐘型）に近づくか確認してください。

第 7 回 標本分布

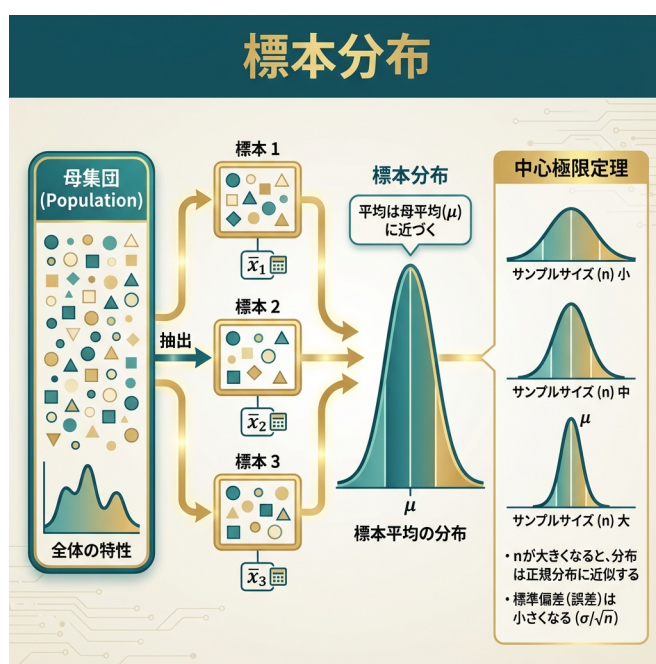


Figure7: 標本分布

一滴の血液で、全身を知る

たった数 ml の血液を検査するだけで、全身の健康状態がわかってしまう。選挙の出口調査では、数千人の回答から全国民の意思を予測できる。

なぜ、そんなことができるのか？ なぜ、「一部」を見るだけで「全体」が見えるのか？

今回は、統計的推測の根幹を支える**標本分布**という考え方と、そこに働く数学的な魔法（大数の法則・中心極限定理）について学びます。これで、記述統計から推測統計への架

け橋がかかる。

1. 未知の世界と鍵穴：母集団と標本

私たちが本当に知りたいのは「全体」のこと。しかし、手に入るのは常に「一部」のデータだけ。

- **母集団 (Population):** 知りたい対象の全体。「未知の世界」。無限の広がりを持つ確率分布としてモデル化されます。
- **標本 (Sample):** そこから切り取られた一部のデータ。「鍵穴から覗いた景色」。推測の、唯一の手がかり。

全数調査の限界

母集団をすべて調べる「全数調査」は、コスト的にも時間的にもほぼ不可能。すべての製品を破壊検査するわけにはいかないし、全ての消費者にアンケートを取ることもできない。だからこそ、標本から母集団を推測する技術（推測統計学）が必要になるのです。

ランダムという名の公平性

標本が母集団を正しく映し出すためには、**ランダム標本**でなければならない。恣意的に選ばれたデータではなく、確率 $F(\mu, \sigma^2)$ に従って選ばれた独立なデータたち。これを数学的には i.i.d. (独立同一分布) と呼びます。

2. パラレルワールドの実験：標本分布

ここから少し、想像力を働かせてほしい。もし、同じ母集団から、何度も何度も標本抽出をやり直したとしたら、どうなるか？

今回の調査では平均値が 50 だった。でも別の調査（パラレルワールド）では 52 かもしれないし、48 かもしれない。この「統計量（平均値など）のばらつき」を表した確率分布を、**標本分布 (Sampling Distribution)** と呼びます。

現実には調査は1回しかできません。しかし、この「もし何度もやったらどうなるか」という仮想的な分布こそが、推測の精度の鍵を握っています。

Python でパラレルワールドをシミュレーションしてみましょう。

```
import numpy as np

rng = np.random.default_rng(1)

# 母集団分布（平均 50, 標準偏差 10 の正規分布）
population = rng.normal(loc=50, scale=10, size=100_000)

# パラレルワールド 1: 1 回目の抽出 (n=100)
sample1 = rng.choice(population, size=100, replace=False)
sample1.mean()

# パラレルワールド 2: もう 1 回抽出
sample2 = rng.choice(population, size=100, replace=False)
sample2.mean()
```

49.81572450065704

毎回結果が少しずつ違う。この「ブレ」を数学的に捉えるのが目標です。

3. 数学的な魔法

サンプルサイズ（標本の大きさ）が増えると、2つの重要な現象が起きる。

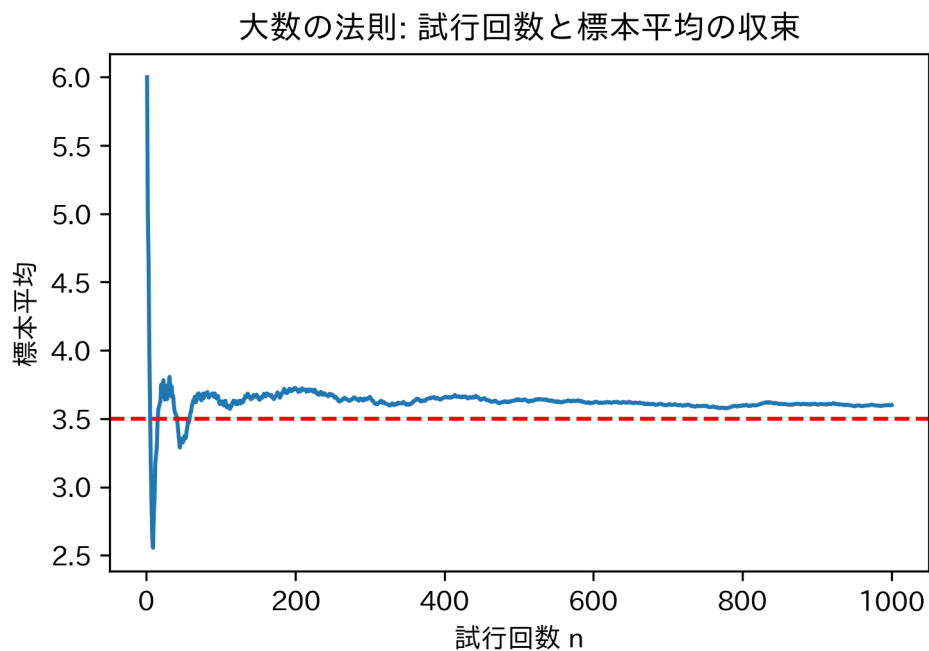
大数の法則：真実への収束

サンプルサイズ n を大きくしていくと、標本平均は母平均（真の値）に限りなく近づいていく。「数は力なり」。データが多ければ多いほど、誤差は消えていく。

```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

rng = np.random.default_rng(0)
n = 1000
dice_rolls = rng.integers(1, 7, size=n)
cumulative_mean = np.cumsum(dice_rolls) / np.arange(1, n + 1)

plt.plot(np.arange(1, n + 1), cumulative_mean)
plt.axhline(3.5, color="red", linestyle="--")
plt.title("大数の法則: 試行回数と標本平均の収束")
plt.ylabel("標本平均")
plt.xlabel("試行回数 n")
plt.show()
```



中心極限定理 (CLT)：混沌からの秩序

ここが統計学のハイライト。母集団がどんなに歪な分布であっても、サンプルサイズ n が大きければ、その標本平均の分布は正規分布に近づいていく。

元のデータが正規分布していなくても、平均値の分布は正規分布になる。この定理のおかげで、私たちはあらゆるデータに対して正規分布を前提とした推測を行うことができる。

実験：一様分布から正規分布へ母集団として平坦な一様分布を用意し、そこから標本平均を計算する実験を行います。

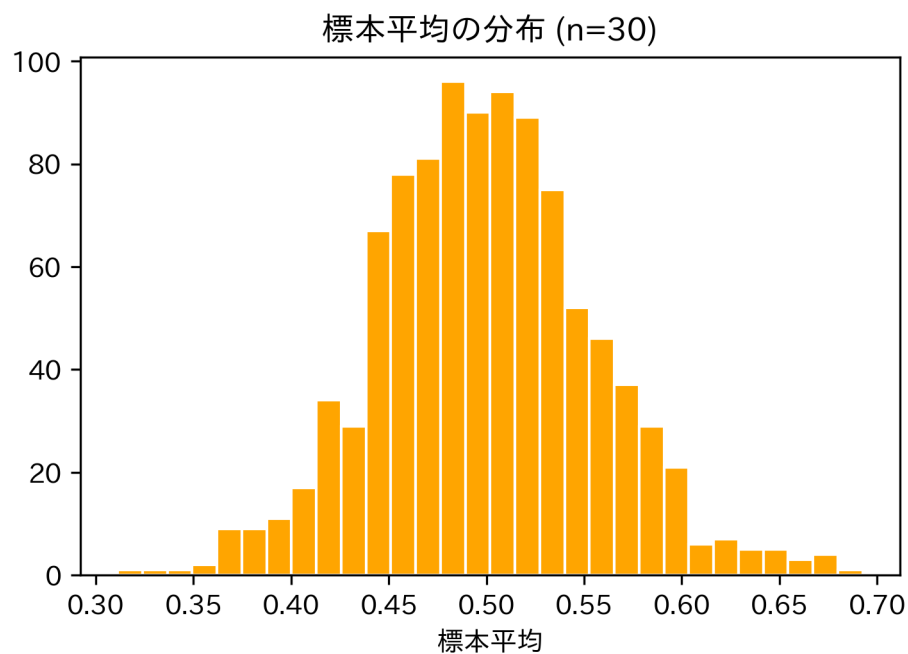
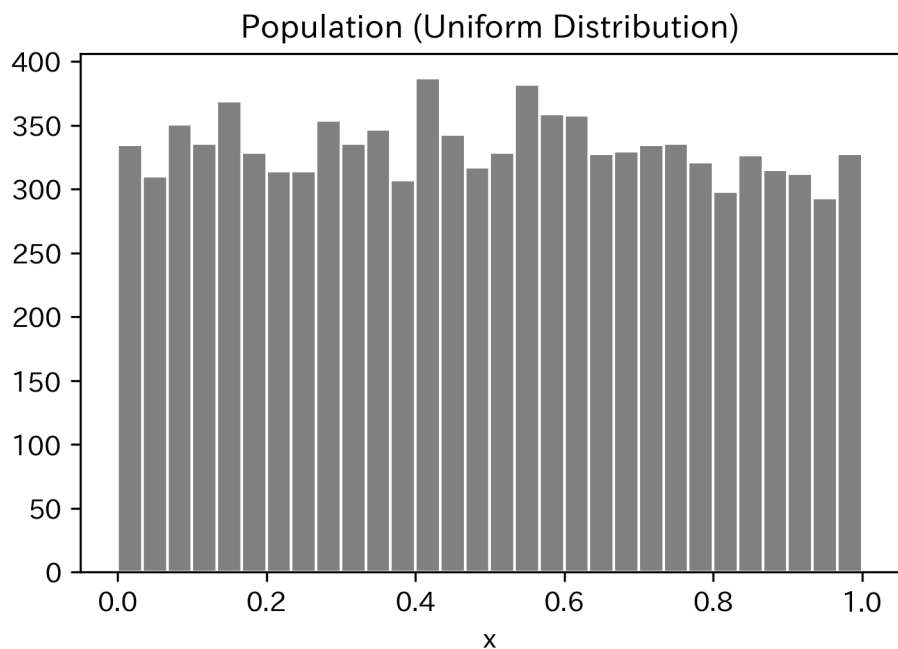
```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

rng = np.random.default_rng(123)

# 1. 一様分布のヒストグラム（母集団の形状）
population = rng.random(10_000)
plt.hist(population, bins=30, color="gray", edgecolor="white")
plt.title("Population (Uniform Distribution)")
plt.xlabel("x")
plt.show()

# 2. 標本平均の分布（CLT の確認）
n_samples = 1000
n = 30 # サンプルサイズ
means = rng.random((n_samples, n)).mean(axis=1)

plt.hist(means, bins=30, color="orange", edgecolor="white")
plt.title("標本平均の分布 (n=30)")
plt.xlabel("標本平均")
plt.show()
```



見事に釣鐘型（正規分布）が現れました。これが中心極限定理の威力。

サンプルサイズと分布の鋭さ

データが増えると、分布はどう変化するのか。

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

rng = np.random.default_rng(123)

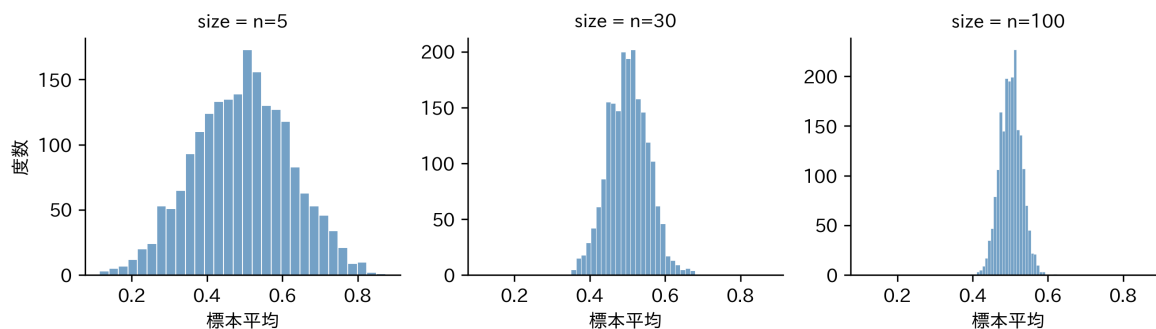
sample_sizes = [5, 30, 100]
n_samples = 2000

rows = []
for n in sample_sizes:
    means = rng.random((n_samples, n)).mean(axis=1)
    rows.append(pd.DataFrame({"means": means, "size": f"n={n}"}))

results = pd.concat(rows, ignore_index=True)

g = sns.FacetGrid(results, col="size", col_order=[f"n={n}" for n in sample_sizes], sharex=True)
g.map_dataframe(sns.histplot, x="means", bins=30, color="steelblue", edgecolor="white")
g.set_axis_labels("標本平均", "度数")
g.fig.suptitle("中心極限定理: サンプルサイズによる標本分布の変化")
g.fig.tight_layout()
plt.show()
```

中心極限定理: サンプルサイズによる標本分布の変化



n が増えるにつれて、分布は正規分布に近づくだけでなく、その幅（ばらつき）が小さくシャープになっていく。

4. 信用の尺度：標準誤差

標本平均の分布のばらつき、つまり「推測の精度」を表す指標が標準誤差 (Standard Error, SE)。

$$SE(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

この式は非常に重要なことを教えてくれる。精度を上げる (SE を小さくする) には、サンプルサイズ n を大きくすればいい。ただし、 $\sqrt{n}$ で効いてくるため、精度を 2 倍にするにはサンプルを 4 倍にする必要があります。

```
import numpy as np

# 理論上の SE (母分散が 1/12 の一様分布 U(0,1) から、n=30 の平均)
true_se = np.sqrt((1 / 12) / 30)

# シミュレーションの SE (標本平均の標準偏差)
sim_se = results.loc[results["size"] == "n=30", "means"].std(ddof=1)

print("理論的 SE:", true_se)
print("シミュレーション SE:", sim_se)
```

理論的 SE: 0.05270462766947299

シミュレーション SE: 0.05392925917835348

標準偏差 (SD) vs 標準誤差 (SE)

この 2 つは混同しやすいので注意してください。

- 標準偏差 (SD): データのばらつき。「個々のデータがどれくらい散らばっている

るか」。

- **標準誤差 (SE):** 平均値のばらつき。「標本平均が真の平均からどれくらいズレるか」。

私たちが知りたい「推定の信用度」を表すのは、標準誤差のほうです。

課題

1. **大数の法則:** コイン投げ（表=1, 裏=0）のシミュレーションを行い、試行回数 n を 10 から 10000 まで増やしたとき、標本平均が理論値 0.5 に収束する様子をグラフで描画してください。
2. **標準誤差の性質:** 母標準偏差 $\sigma = 10$ の正規分布において、標準誤差を 1 以下にするためには、最低何個のサンプルサイズが必要か計算してください。

練習問題

練習 1: 指数分布の CLT

指数分布 (`scipy.stats.expon`) は右に裾が長い歪んだ分布です。この分布からサンプルを抽出しても、標本平均の分布は正規分布に近づくか、シミュレーションで確認してください。

練習 2: サンプルサイズと精度

標準誤差の式 $SE = \sigma / \sqrt{n}$ に基づき、以下の問いに答えてください。

1. サンプルサイズを 100 から 400 に増やすと、標準誤差はどう変化しますか？
2. 標準誤差を $1/10$ にするためには、サンプルサイズを何倍にする必要がありますか？

練習 3: 記述統計と推測統計

手元に $n = 100$ のデータがあります。- このデータの平均値と標準偏差を計算すること（記述統計）と、- このデータの平均値から標準誤差を計算して信頼区間を求めること（推測統計）の違いを、自分の言葉で説明してください。

まとめ

概念	記号/式	意味
標本分布	-	統計量（平均など）のパラレルワールドでの分布
大数の法則	$n \rightarrow \infty$ で $\bar{X} \rightarrow \mu$	データが増えれば平均は真の値に近づく
中心極限定理	n 大なら $\bar{X} \sim N(\mu, \sigma^2/n)$	平均値の分布は（元が何であれ）正規分布になる
標準誤差	$\sigma/\sqrt{n}$	推定の精度。「平均値のブレ幅」

第 8 回統計的推定

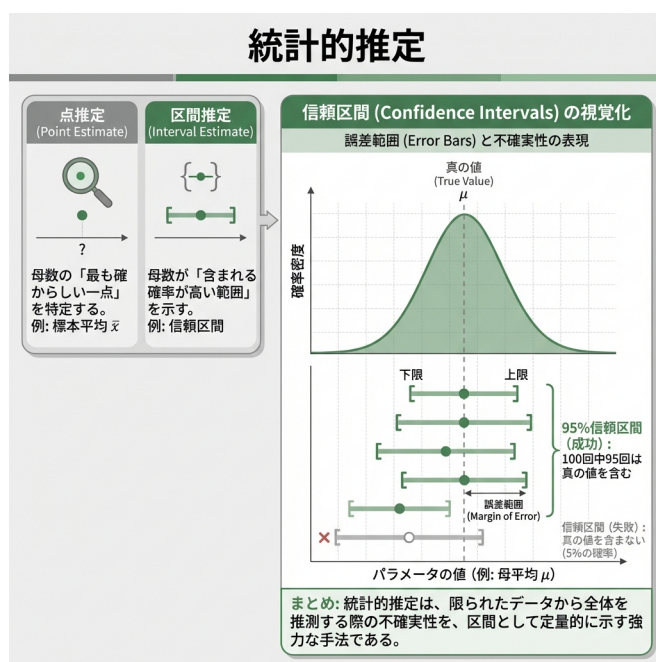


Figure8: 統計的推定

「だいたい」を科学する

「有権者の支持率は 45%」「新薬の効果は従来より 20% 高い」

ニュースでよく聞く、これらの数字。もちろん、有権者全員に聞いたわけでも、世界中の患者に試したわけでもありません。一部のデータ（標本）から、全体（母集団）の真実を推測したもの。

しかし、なぜ「45%」と言い切れるのか？ もし本当は「40%」だったら？ あるいは「50%」

だったら？ その「ズレ」のリスクを、私たちはどう見積もればいいのか？

今回は、統計学の核心部分である**推定 (Estimation)** について学びます。一点張りで予想する「点推定」と、幅を持たせて安全策をとる「区間推定」。この 2 つの武器を使いこなしましょう。

1. 推定の基礎：レシピと料理

まず、言葉の定義をはっきりさせておきましょう。統計的推定には、似て非なる 2 つの用語が登場します。

1. **推定量 (Estimator)**: 計算の**レシピ** (関数)。
 - 「データを全部足して人数で割る」という計算方法そのもの。確率的に変動します。
2. **推定値 (Estimate)**: 実際に作られた**料理** (数値)。
 - 「平均値は 50.3 だった」という結果。確定した数値です。

良いレシピの条件

美味しい料理 (正確な推定値) を作るためには、良いレシピ (優れた推定量) が必要。統計学では、以下の要素を重視します。

- **不偏性**: 「平均的には」正解を指すこと。
- **一致性**: データが増えれば増えるほど、正解に近づいていくこと。

2. 点推定：一点豪華主義

母数 (知りたい真の値) を、たった一つの値で言い当てる方法です。例えば、「日本の平均年収は 450 万円だ!」と言い切るのが点推定。

Tips データセットを使って、チップの支払総額の平均を点推定してみましょう。

```
import pandas as pd
```



```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)

# 点推定（標本平均）
tips["total_bill"].mean()
```

19.78594262295082

答えは 19.78594。これが、現時点での私たちの「ベストな予想」。しかし、これだけでは不十分です。「で、その予想はどれくらい信用できるの？」という問いに答えられないから。

3. 区間推定：輪投げのゲーム

そこで登場するのが**区間推定**。「平均は 19.78 ドルです」と言う代わりに、「平均は 18.66 ドルから 20.91 ドルの間にある確率が高い」と、幅を持たせて答える。

これがいわゆる**信頼区間 (Confidence Interval)** です。

95% 信頼区間の本当の意味

よくある誤解：「真の平均値がこの区間に入る確率は 95% だ」正しくは：「この方法で輪投げ（区間推定）を 100 回やったら、そのうち 95 回は**的（真の平均値）**が入る」

的（母数）は動きません。動くのは、私たちが投げる輪（信頼区間）のほう。

シミュレーションで、実際に輪投げを 100 回やってみましょう。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import t

rng = np.random.default_rng(123)
```

```
# ゲームの設定
n_sim = 100
n_sample = 30
mu = 50      # 的の位置 (真の母平均)
sigma = 10   # 手ブレの大きさ (母標準偏差)
conf_level = 0.95
alpha = 1 - conf_level

rows = []
for i in range(1, n_sim + 1):
    # 1 回の輪投げ (標本抽出)
    x = rng.normal(loc=mu, scale=sigma, size=n_sample)

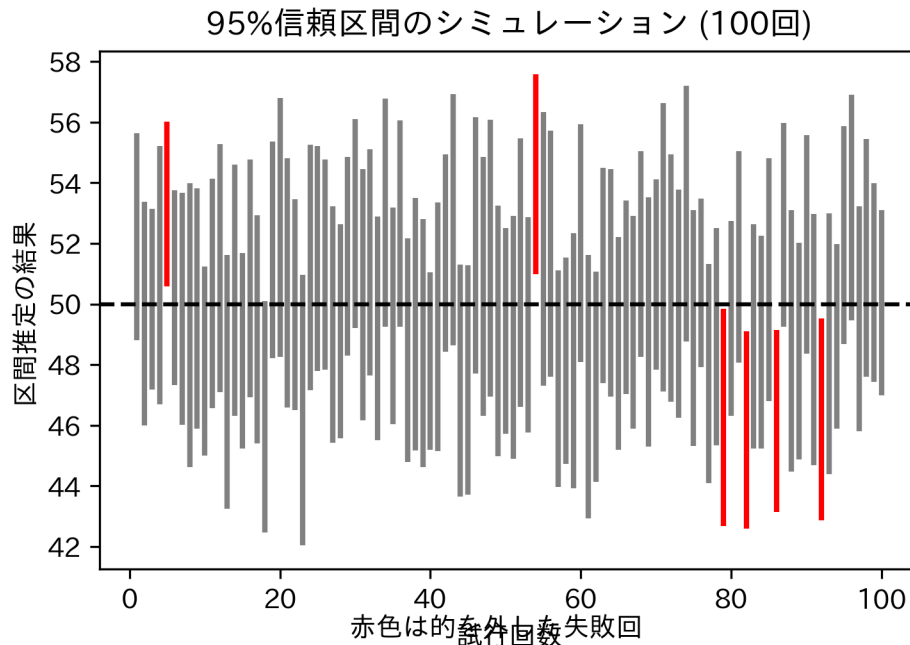
    # 95% 信頼区間 (母分散未知 → t 分布)
    xbar = x.mean()
    s = x.std(ddof=1)
    crit = t.ppf(1 - alpha / 2, df=n_sample - 1)
    half = crit * s / np.sqrt(n_sample)
    lower = xbar - half
    upper = xbar + half

    hit = (mu >= lower) and (mu <= upper)
    rows.append({"id": i, "lower": lower, "upper": upper, "hit": hit})

results = pd.DataFrame(rows)

# 結果発表
colors = results["hit"].map({True: "gray", False: "red"})
plt.vlines(results["id"], results["lower"], results["upper"], colors=colors, line
plt.axhline(mu, linestyle="--", color="black")
plt.title("95% 信頼区間のシミュレーション (100 回)")
```

```
plt.xlabel("試行回数")
plt.ylabel("区間推定の結果")
plt.figtext(0.5, 0.01, "赤色は的を外した失敗回", ha="center")
plt.show()
```



ご覧の通り、いくつかの赤い線（失敗）があります。「95% 信頼区間」とは、「長期的に見ればこれくらいの失敗率（5%）に収まるようなルールで区間を決めた」という保証書のようなもの。

4. t 分布：不確実性のペナルティ

信頼区間を計算するとき、データ数が少ない場合は **t 分布** という分布を使います。

これは「標準正規分布」によく似ていますが、少しだけ裾が広がっています。なぜか？

母分散（真のばらつき）がわからないため、代わりに標本から計算した不偏分散を使うからです。「本当のばらつきがわからない」という不確実性の分だけ、区間（輪の大きさ）を少し広めにとって安全策をとる。それが、t 分布の役割。

データ数（サンプルサイズ）が増えれば、この不確実性は減り、t 分布は正規分布に近づ

いていく。

Python で実践：平均の区間推定

母分散が未知のとき、平均の信頼区間は「標本平均 $\pm$ t 臨界値 $\times$ 標準誤差」で計算できます。

```
# 支払総額の 95% 信頼区間
import numpy as np
from scipy.stats import t

x = tips["total_bill"].to_numpy()
n = len(x)
xbar = x.mean()
s = x.std(ddof=1)
crit = t.ppf(0.975, df=n - 1)
half = crit * s / np.sqrt(n)
(xbar - half, xbar + half)
```

```
(18.663331704358473, 20.908553541543167)
```

95 percent confidence interval の行を見てください。これが、「今回の輪投げの結果」です。

信頼区間の幅を決める 3 要素

輪の大きさ（信頼区間の幅）は何で決まるのか？

1. **信頼係数**: 「絶対に外したくない (99%)」と思えば、輪は大きくなる。
2. **サンプルサイズ**: データが増えれば、的の位置がはっきり見えるので、輪は小さく（精度良く）なる。
3. **ばらつき**: データが暴れている（分散が大きい）と、安全のために輪を大きくする必要がある。

```
import numpy as np
import pandas as pd
from scipy.stats import t

def mean_ci(x, conf_level: float):
    x = np.asarray(x, dtype=float)
    n = len(x)
    xbar = x.mean()
    s = x.std(ddof=1)
    alpha = 1 - conf_level
    crit = t.ppf(1 - alpha / 2, df=n - 1)
    half = crit * s / np.sqrt(n)
    return xbar - half, xbar + half

levels = [0.90, 0.95, 0.99]
rows = []
for cl in levels:
    lo, hi = mean_ci(tips["total_bill"], cl)
    rows.append({"Confidence_Level": f"{int(cl * 100)}%", "Lower": lo, "Upper": hi})

pd.DataFrame(rows)
```

	Confidence_Level	Lower	Upper
0	90%	18.844923	20.726963
1	95%	18.663332	20.908554
2	99%	18.306313	21.265572

5. 母比率の区間推定

平均値だけでなく、「割合（比率）」も区間推定できます。「内閣支持率は45% ± 3%」といったニュースは、これを使っています。

例：100 人中 60 人が賛成。母集団での賛成率は？

```
from statsmodels.stats.proportion import proportion_confint

count = 60
nobs = 100
phat = count / nobs
ci = proportion_confint(count, nobs, alpha=0.05, method="wilson") # スコア (Wilson)
{"p_hat": phat, "ci_95": ci}
```

```
{'p_hat': 0.6, 'ci_95': (0.5020025867910618, 0.6905987135675411)}
```

課題

1. **Tips データの分析:** tips データの tip (チップ額) について、平均値の 99% 信頼区間を求めてください。
2. **解釈の確認:** 算出された区間について、「チップの母平均が 99% の確率でこの値になる」という説明がなぜ誤りなのか、論じてください。

練習問題

練習 1: 信頼区間の幅とサンプルサイズ

記述統計的に同じ平均と分散を持つデータでも、サンプルサイズが異なると信頼区間はどう変わるか確認してください。(numpy.random.Generator.normal で $n = 10$ と $n = 100$ のデータを生成し、t 分布を用いた信頼区間の式で比較)

練習 2: 母比率のシミュレーション

真の支持率が 50% のとき、100 人の世論調査を 100 回繰り返したと仮定し、95% 信頼区間が 0.5 を含まない回数が何回あるかシミュレーションしてください。

練習 3: 記述統計と推測統計

手元に $n = 100$ のデータがあります。

- 「データの平均」と「母平均の点推定値」は値としては同じですが、概念としてどう異なりますか？

まとめ

概念	説明	Python での実装例
推定量	計算のレシピ（関数）。	-
推定値	出来上がった料理（数値）。	-
信頼区間	輪投げの輪。的（母数）を捉える確率的範囲。	<code>scipy.stats.t.ppf</code> + 式、 <code>statsmodels.stats.proportion.proportion</code>
t 分布	データ不足の不確実性を考慮した、少し幅広の正規分布。	<code>scipy.stats.t</code>

第 9 回 仮説検定 (1)

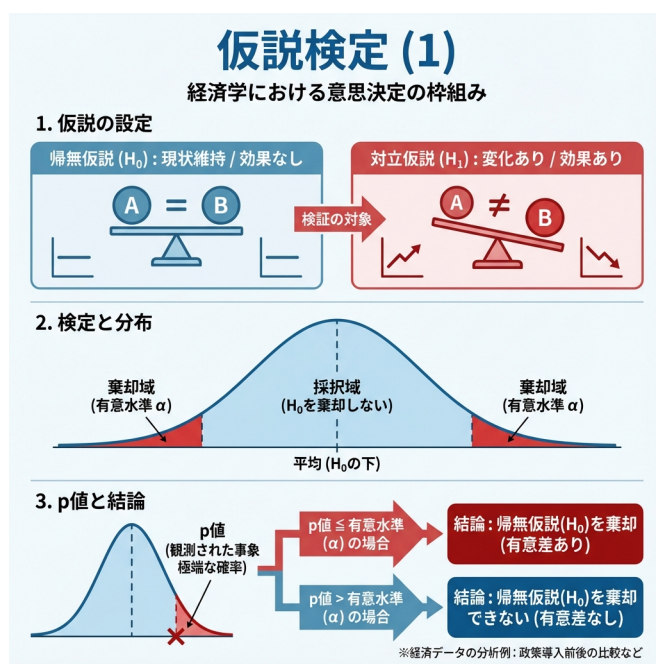


Figure9: 仮説検定 (1)

裁判官になるための統計学

「この新薬は効く!」「私の予知能力は本物だ!」

世の中には、様々な主張が溢れている。しかし、科学の世界では「言った者勝ち」は通用しない。その主張が、たまたまの偶然なのか、それとも確かな証拠に基づいているのか。それを厳格にジャッジする手続きが**仮説検定 (Hypothesis Testing)**。

仮説検定の仕組みは、**刑事裁判**にそっくりです。今回は、あなたが裁判官となって、「偶然かどうか」という難事件に判決を下す方法を学ぼう。

1. 裁判の基本ルール：疑わしきは罰せず

統計的検定の世界では、まず最初に「何の効果もない（無罪）」と仮定することから始めます。これを覆すだけの強力な証拠が出て初めて、「効果がある（有罪）」と認めることになる。

登場人物（仮説）

1. 被告人：帰無仮説 (H_0)
 - 「差はない」「効果はない」という、否定したい仮説。
 - 裁判で言えば「被告人は無罪である」という前提。
2. 検察官：対立仮説 (H_1)
 - 「差がある」「効果がある」という、証明したい仮説。
 - 裁判で言えば「被告人は有罪である」という主張。

検定のゴールは、「無罪 (H_0) と仮定するには、証拠（データ）が不自然すぎる」ことを示し、無罪の前提を棄却すること。

2. 証拠の評価：p 値という驚き

裁判官であるあなたは、提出された証拠（データ）を見て判断する。ここで重要なのが **p 値 (p-value)**。

p 値とは、「もし被告人が無罪だとしたら、こんな証拠が出てくる確率はどれくらいか？」という値。

- **p 値が大きい (0.5 とか)**：「無罪の人からも、よく出る証拠だ」。
 - 判決：無罪を棄却できない（証拠不十分）。
- **p 値が極めて小さい (0.001 とか)**：「無罪だとしたら、こんな証拠が出るのはありえない（奇跡に近い）」。

－ 判決：無罪ではないだろう。有罪（有意）！

判決の基準：有意水準

どこまで確率が低ければ「ありえない」と判断するのか？ その基準を**有意水準**と呼び、通常 **5% (0.05)** に設定します。

- 「無罪なのにこんな証拠が出る確率は 5% 未満」なら、それはもう偶然とは考えない。
- この基準を超えたとき、「統計的に有意である (Statistically Significant)」と宣言します。

3. シミュレーション：無罪の証明

t 分布を使って、「無罪（偶然）」の範囲を可視化してみましょう。

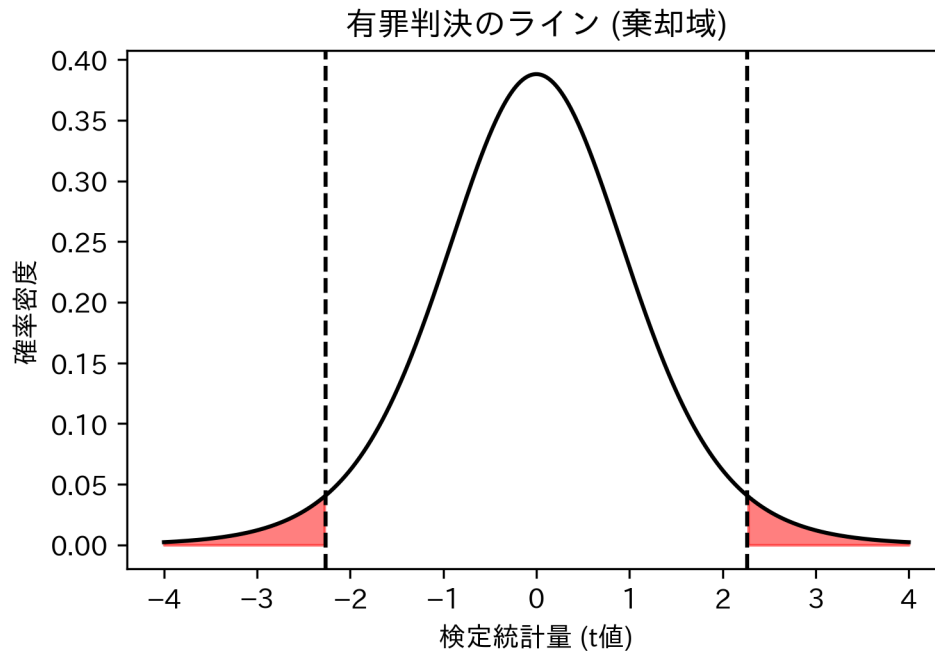
```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import t

df = 9
x = np.linspace(-4, 4, 400)
y = t.pdf(x, df=df)

# 判決のライン（有意水準 5% の両側）
critical_val = t.ppf(0.975, df=df)

plt.plot(x, y, color="black")
plt.fill_between(x, 0, y, where=(x > critical_val), color="red", alpha=0.5)
plt.fill_between(x, 0, y, where=(x < -critical_val), color="red", alpha=0.5)
plt.axvline(critical_val, linestyle="--", color="black")
plt.axvline(-critical_val, linestyle="--", color="black")
```

```
plt.title("有罪判決のライン (棄却域)")
plt.xlabel("検定統計量 (t 値)")
plt.ylabel("確率密度")
plt.show()
```



赤いエリアが「有罪（棄却域）」。データから計算した t 値がここに入れば、「無罪」の前提は棄却されます。

4. 実践：1 標本の t 検定

実際の事件（データ）を裁いてみましょう。

事件: あるクラス 10 人のテスト平均点は、全国平均の「50 点」と違うのか？ `scores = [55, 60, 45, 70, 58, 62, 48, 52, 65, 50]`

1. 帰無仮説 (H_0): 平均点に違いはない ($\mu = 50$)。
2. 対立仮説 (H_1): 平均点に違いがある ($\mu \neq 50$)。

```
import numpy as np
from scipy.stats import ttest_1samp

scores = np.array([55, 60, 45, 70, 58, 62, 48, 52, 65, 50])
ttest_1samp(scores, popmean=50)
```

TtestResult(statistic=2.5862432897084537, pvalue=0.029394044701776194, df=9)

判決: p 値は 0.02939。これは 0.05 (5%) より小さい。つまり、「平均 50 点の集団から、偶然この点数が取れる確率は約 2.9% しかない」。裁判官（あなた）は、「偶然にしては出来すぎている」と判断し、**帰無仮説を棄却**します。

結論: このクラスの平均点は、全国平均と有意に異なる。

5. 冤罪のリスク：2つの過誤

裁判に誤審があるように、統計的検定にも誤りのリスクがあります。

	被告人は無実 (H_0 真)	被告人はクロ (H_1 真)
有罪判決 (H_0 棄却)	冤罪 (第 1 種の過誤) 何も無いのに「ある」と言う 確率 α (5%)	正しい判決
無罪判決 (H_0 棄却せず)	正しい判決	取り逃がし (第 2 種の過誤) あるのに「ない」と言う 確率 β

我々が設定した「有意水準 5%」とは、「5% の確率で冤罪（第 1 種の過誤）を起こすことを許容する」という宣言でもある。だからこそ、p 値が小さいことは「絶対の真実」ではなく、「かなり確からしい」という程度の意味なのだ。

課題

1. **Tips データの分析:** tips データの total_bill の平均値が 20 ドル と異なるかどうかを検定してください。帰無仮説 H_0 と対立仮説 H_1 を数式で記述し、結果の p 値に基づいて結論（棄却するか否か）を述べてください。

練習問題

練習 1: 結論の解釈

ある検定の結果、p 値が 0.08 でした。有意水準 5% の場合、以下の主張は正しいですか？「p 値 > 0.05 なので、帰無仮説は正しいと証明された。」

練習 2: サンプルサイズと有意差

前回使用したシミュレーションコードを使い、 $H_0: \mu = 50$ が誤り（真の $\mu = 52$ ）であるデータを生成してください。サンプルサイズ n を変えたとき、p 値がどう変化するか確認してください。

練習 3: 誤りの種類

「ガンではない患者を、誤ってガンであると診断してしまう」のは、第 1 種の過誤ですか、第 2 種の過誤ですか？（帰無仮説：「ガンではない」とした場合）

まとめ

裁判用語	統計用語	意味
被告人は無罪	帰無仮説 (H_0)	「差はない」という前提
被告人は有罪	対立仮説 (H_1)	「差がある」という主張
証拠の不自然さ	p 値	無罪前提でそのデータが出る確率

裁判用語	統計用語	意味
判決の基準	有意水準 (α)	p 値がこれ未満なら有罪 (棄却)
冤罪	第 1 種の過誤	無罪なのに有罪にしてしま うミス

第 10 回 仮説検定 (2)

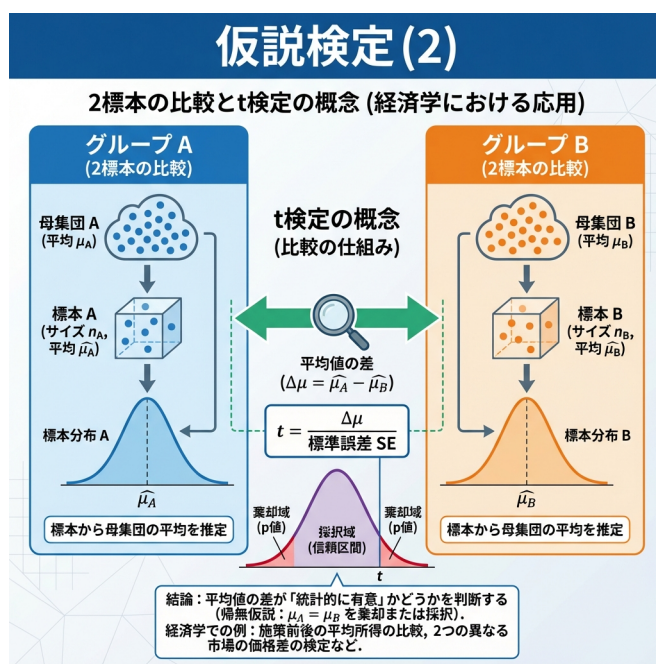


Figure10: 仮説検定 (2)

A vs B : 決着をつけよう

「新薬 A は、プラセボ B より効くのか?」「新しい教育プログラムを受けた生徒は、受けていない生徒より成績が良いか?」「ダイエット前後で、体重は本当に減ったのか?」

私たちの世界は、比較で溢れている。しかし、単に平均値を比べるだけでは不十分。偶然のバラつきかもしれないからだ。

今回は、2つのグループの間に「意味のある差」があるかどうかを判定する、統計学の定番技 **2 標本の t 検定** をマスターしよう。

1. 2つの島を比べる：対応のない t 検定

まずは、全く別々の2つのグループを比較するケース。例えば、「ランチ島」に住む人々と、「ディナー島」に住む人々。彼らの支払金額に差はあるのか？

このように、互いに独立したデータを比較する手法を**対応のない t 検定 (Unpaired t-test)**と呼ぶ。

地形を選ばない万能車：Welch の t 検定

2つの島（グループ）は、地形（データのばらつき＝分散）が同じとは限らない。昔の教科書には「まず地形が同じか調べて（F 検定）、それから車を選べ」と書いてあった。

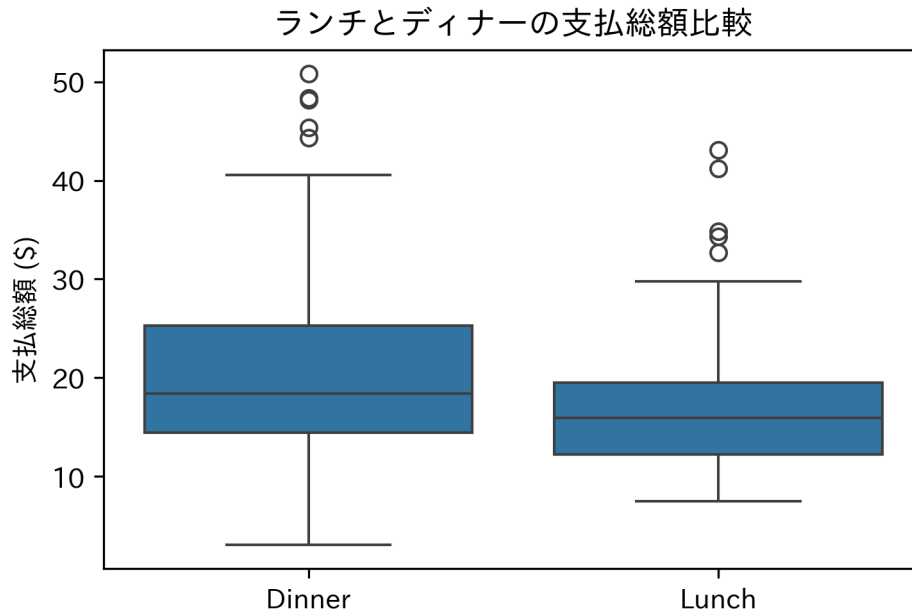
しかし、現代の統計学では **Welch の t 検定** という「全地形対応車」を最初から使うのが常識。これなら、分散が違っていても安全に走行（検定）できる。

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401
from scipy.stats import ttest_ind

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)

# 箱ひげ図：2つの島の地形確認
ax = sns.boxplot(data=tips, x="time", y="total_bill")
ax.set(title="ランチとディナーの支払総額比較", xlabel="", ylabel="支払総額 ($)")
plt.show()
```

```
# Welch の t 検定 (equal_var=False)
dinner = tips.loc[tips["time"] == "Dinner", "total_bill"]
lunch = tips.loc[tips["time"] == "Lunch", "total_bill"]
ttest_ind(dinner, lunch, equal_var=False)
```



TtestResult(statistic=3.122986183296264, pvalue=0.002166573514803894, df=143.29265556532)

判定: p 値は約 0.002。これは「差がないとしたら、こんなデータは 0.2% の確率でしか起きない」ことを意味する。偶然とは考えにくい。つまり、ランチとディナーには有意な差があると結論づける。

2. ビフォー・アフター：対応のある t 検定

次は、同じ人が「変身」するケース。ダイエット前とダイエット後。テスト勉強前と勉強後。

ここでは、グループ全体の平均を見るのではなく、「一人ひとりの変化（差分）」に注目する。これを**対応のある t 検定 (Paired t-test)** と呼ぶ。

変化を捕まえる

同じ人のデータを追跡することで、個人の「もともとの性質」をキャンセルし、純粋な「変化」だけを取り出すことができる。

```
import numpy as np
from scipy.stats import ttest_rel

# 5 人のダイエット記録
before = np.array([70, 80, 75, 60, 65])
after  = np.array([68, 78, 74, 58, 64])

ttest_rel(before, after)
```

TtestResult(statistic=6.531972647421809, pvalue=0.0028378459267344464, df=4)

事実確認: もしこれを「対応なし (別人のデータ)」として分析してしまうと、p 値は 0.75 になり、「差なし」と判定されてしまう。対応のある検定を使うことで初めて、微小な変化 (-1.6kg) を「有意」として検出できるのだ。

3. その差は、どれくらい凄いのか？ : 効果量

p 値が小さいことは、「偶然ではない」ことを保証するだけ。「すごい差がある」ことの証明ではない。

1 万人調査すれば、0.1 ミリの身長差でも「有意差あり ($p < 0.05$)」になる。でも、0.1 ミリに意味はあるか？

そこで必要なのが、差の実質的な大きさを表す「スコア」、すなわち**効果量 (Effect Size)**。代表的なスコアである **Cohen's d** を使おう。

シグナルの強さを測る

$$Cohen's\ d = \frac{\text{平均値の差}}{\text{標準偏差 (ばらつき)}}$$

これは、「データのばらつき（ノイズ）」に対して、「平均値の差（シグナル）」がどれくらい突き抜けているかを表す。

- **0.2:** 小さい（ノイズに埋もれがち）
- **0.5:** 中くらい（肉眼でもなんとなく分かる）
- **0.8:** 大きい（誰が見ても明らか）

```
import numpy as np

def cohens_d(x, y):
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    nx = len(x)
    ny = len(y)
    sx = x.std(ddof=1)
    sy = y.std(ddof=1)
    s_pooled = np.sqrt(((nx - 1) * sx**2 + (ny - 1) * sy**2) / (nx + ny - 2))
    return (x.mean() - y.mean()) / s_pooled

# ランチとディナーの差のスコア (Cohen's d)
cohens_d(dinner, lunch)
```

0.41374063784312753

スコアは約 0.41。中程度の効果だ。「この差は統計的に有意であり、実質的にも中程度の意味がある」と、胸を張って報告しよう。

課題

1. 喫煙の有無とチップ: tips データを使って、喫煙者 (Smoker) と非喫煙者 (Non-Smoker) で、チップの額 (tip) に差があるかを検定してください (Welch の t 検定)。また、その差の効果量 (Cohen's d) を計算し、統計的有意性と実質的意味について論じてください。

練習問題

練習 1: 検定の選択

以下のシナリオにおいて、最も適切な検定手法 (対応あり t 検定 / 対応なし (Welch)t 検定) を選んでください。

1. A 組 30 人と B 組 30 人のテストの平均点の比較。
2. 高血圧患者 30 人の、降圧剤投与前と投与後の血圧の比較。
3. 10 組の双子における、兄と弟の身長の比較。

練習 2: サンプルサイズと p 値

効果量 $d = 0.5$ (中程度) の差がある 2 つの母集団から、(a) $n = 10$ ずつ (b) $n = 100$ ずつデータを抽出し、t 検定を行った場合の p 値をシミュレーションで比較してください。サンプルサイズが p 値に与える影響を確認します。

練習 3: 等分散性の確認はいらない？

「まず F 検定をしてから t 検定の種類を選ぶ」という手順が、現在では推奨されない理由を自分の言葉で説明してください。

まとめ

検定手法	適用場面	Python コード	備考
Welch の t 検定	独立した 2 群の比較	<code>scipy.stats.ttest_ind(x, y, equal_var=False)</code>	分散を仮定しない。
対応のある t 検定	同一対象・ペアの比較	<code>scipy.stats.ttest_rel(before, after)</code>	差の t 検定と等価。検出力が高い。
効果量 (Cohen's d)	差の実質的な大きさ	(計算式参照)	サンプルサイズに依存しない指標。

第 11 回相関と単回帰分析

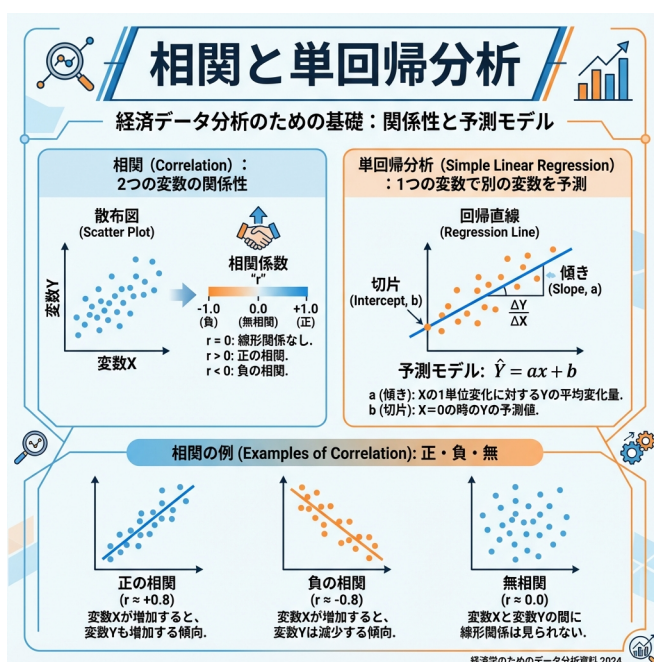


Figure11: 相関と単回帰分析

未来を予測する魔法

「もっと勉強したら、成績はどれくらい上がる?」「気温が1度上がったら、ビールは何本多く売れる?」

もし、あるデータ (X) を知ることで、未知のデータ (Y) を予測できるとしたら? それはビジネスでも科学でも、最強の武器になる。

今回は、2つのデータの関係性を読み解き、未来を予測するための数理モデル**単回帰分析**

を習得しよう。

1. 2 人のダンス：相関 (Correlation)

予測の第一歩は、「2 つのデータに関係があるか？」を知ること。これを**相関**と呼ぶ。

イメージしてほしい。2 人のダンサーが踊っている。片方が右に動いたとき、もう片方も必ず右に動くなら、2 人の息はピッタリだ（強い正の相関）。バラバラに動いているなら、関係はない（無相関）。

この「息の合い具合」を数値化したのが **相関係数** r 。

- $r = 1$: 完全なシンクロ（正）。
- $r = -1$: 正反対の動き（負）。
- $r = 0$: 全くの無関係。

実践: 支払額とチップの関係

「支払額 (X) が多い客ほど、チップ (Y) も多く払うのか？」を確認してみよう。

```
import pandas as pd
from scipy.stats import pearsonr

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)

# 相関係数の算出
tips["total_bill"].corr(tips["tip"])

# 検定: その相関は偶然ではないか?
pearsonr(tips["total_bill"], tips["tip"])
```

```
PearsonRResult(statistic=0.6757341092113646, pvalue=6.692470646863359e-34)
```

結果は $r \approx 0.68$ 。「かなり息が合っている（強い正の相関）」と言える。支払額が増えれば、チップも増える傾向がはっきりとある。

2. 予測マシンを作る：単回帰分析

相関があることが分かったら、次は具体的に予測をする。「支払いが 10 ドル増えたら、チップはいくら増えるの？」

この問いに答えるために、データの真ん中を貫く「運命の赤い糸（回帰直線）」を引く。この直線の式が、私たちの**予測マシン**になる。

$$\text{予測値} = \text{切片} + \text{傾き} \times \text{支払額}$$

交換レートとしての「傾き」

ここで最も重要なのが **傾き** (β_1)。これは、 X と Y の**交換レート**のようなもの。「支払額を 1 ドル投入すると、チップが何ドル返ってくるか？」を表す。

```
# 予測モデルの作成
import statsmodels.formula.api as smf

model = smf.ols("tip ~ total_bill", data=tips).fit()
model.summary()
```

Dep. Variable:	tip	R-squared:	0.457
Model:	OLS	Adj. R-squared:	0.454
Method:	Least Squares	F-statistic:	203.4
Date:	Tue, 10 Feb 2026	Prob (F-statistic):	6.69e-34
Time:	22:35:46	Log-Likelihood:	-350.54
No. Observations:	244	AIC:	705.1
Df Residuals:	242	BIC:	712.1
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.9203	0.160	5.761	0.000	0.606	1.235
total_bill	0.1050	0.007	14.260	0.000	0.091	0.120

Omnibus:	20.185	Durbin-Watson:	2.151
Prob(Omnibus):	0.000	Jarque-Bera (JB):	37.750
Skew:	0.443	Prob(JB):	6.35e-09
Kurtosis:	4.711	Cond. No.	53.0

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

解析結果: - 傾き (Estimate of total_bill): 0.105

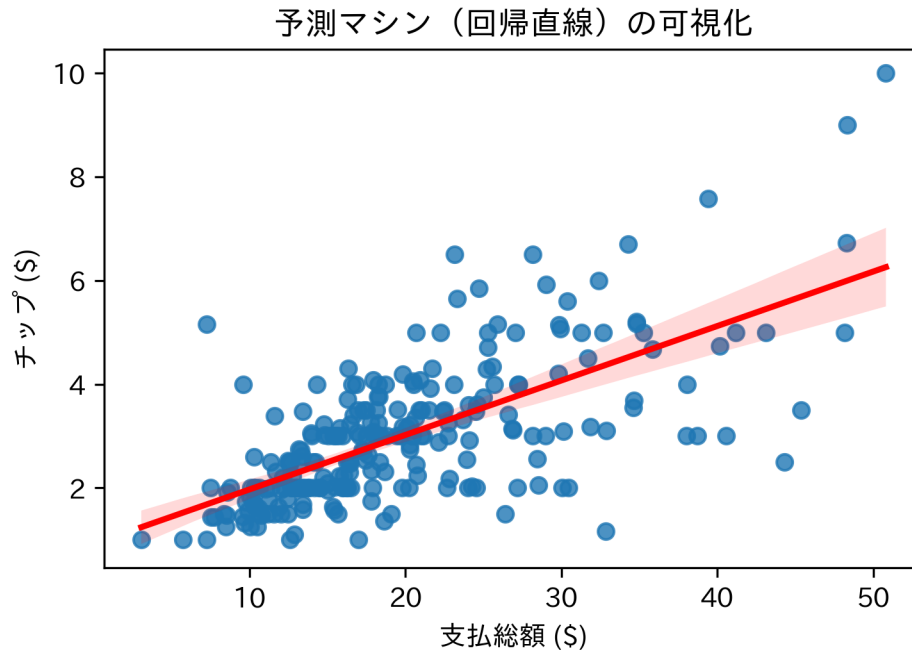
これは、「支払額が 1 ドル増えるごとに、チップは約 10.5 セント増える」という法則を表している。これさえ分かれば、まだ見ぬ客のチップ額も予測できる。

直線の可視化

```
import seaborn as sns
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

ax = sns.regplot(data=tips, x="total_bill", y="tip", line_kws={"color": "red"})
```

```
ax.set(title="予測マシン（回帰直線）の可視化", xlabel="支払総額（$）", ylabel="チップ（$）")  
plt.show()
```



赤い線が、推定された予測モデルだ。

3. モデルの品質チェック：残差診断

モデルを作って終わりではない。「この予測マシンは本当に優秀か？」をチェックする必要がある。

注目するのは、予測と実測のズレ、すなわち **残差 (Residuals)**。完璧なモデルなら、残差は単なる「ランダムなノイズ」になる。もし残差に何らかのパターン（癖）が残っていたら、モデルはまだ何か重要な情報を見落としていることになる。

決定係数: 説明力

まず、このモデルがデータのバラツキの何%を説明できたかを確認する。

- Multiple R-squared: 0.4566

- 「チップの変動の約 46% は、支払額の違いで説明できる」
- 残りの 54% は、支払額以外の要因（客の気前の良さや接客態度など）。

残差の健康診断

モデル変な癖がないか、レントゲン（残差プロット）を撮って確認する。

```
# 予測値と残差の準備
import matplotlib.pyplot as plt
import seaborn as sns
import japanize_matplotlib # noqa: F401
from statsmodels.graphics.gofplots import qqplot

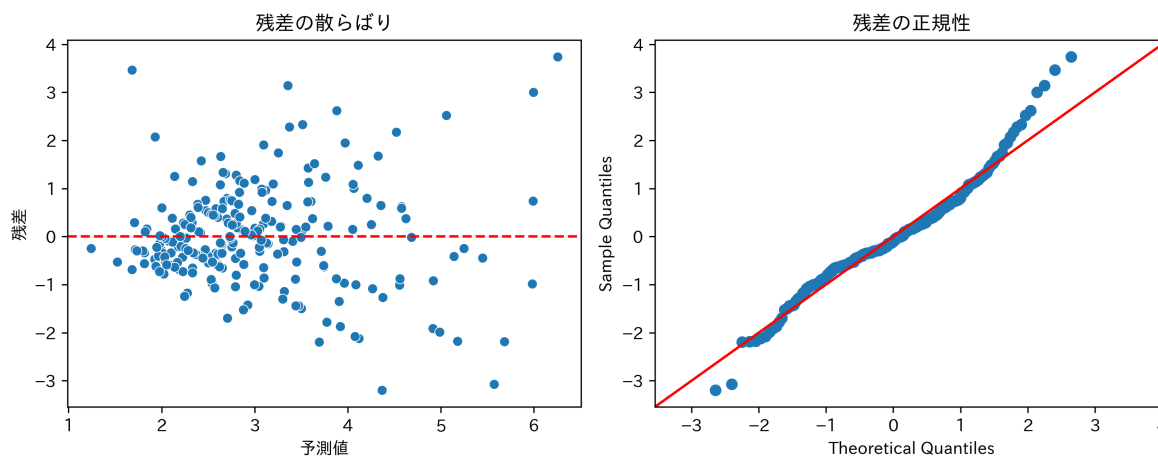
tips_aug = tips.assign(pred=model.fittedvalues, resid=model.resid)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# 1. 残差プロット（クセがないか？）
sns.scatterplot(data=tips_aug, x="pred", y="resid", ax=axes[0])
axes[0].axhline(0, color="red", linestyle="--")
axes[0].set(title="残差の散らばり", xlabel="予測値", ylabel="残差")

# 2. Q-Q プロット（正規分布しているか？）
qqplot(tips_aug["resid"], line="45", ax=axes[1])
axes[1].set(title="残差の正規性")

plt.tight_layout()
plt.show()
```



- 残差プロット: 0 の周りにバランスよく散らばっていれば合格。
- Q-Q プロット: 赤い線の上に並んでいれば合格。

課題

1. 単回帰分析の実践: tips データを用いて、size (客数) を説明変数、 $Y = \text{total_bill}$ (支払総額) を被説明変数とする単回帰分析を行ってください。得られた傾き $\hat{\beta}_1$ の値を解釈し、その統計的有意性について述べてください。

練習問題

練習 1: 相関の罠

架空のデータにおいて、「アイスクリームの売上」と「水難事故の件数」に強い正の相関が見られました。これは「アイスクリームを食べることが水難事故の原因である」ことを意味しますか？ 第三の要因（交絡因子）を挙げて説明してください。

練習 2: 決定係数の解釈

ある回帰モデルの決定係数が $R^2 = 0.05$ でしたが、傾きの検定は $p < 0.001$ で有意でした。この結果はどう解釈すべきですか？「モデルは無意味」ですか、それとも「関係はあるが予測精度は低い」ですか？

練習 3: 残差のパターン

残差プロットにおいて、残差が U 字型のパターンを描いていました。これはモデルの何の仮定に違反している可能性が高いですか？（ヒント：線形性）

まとめ

概念	アナロジー	意味
相関	ダンスのシンクロ率	2 つの変数の動きの連動性。
回帰直線	予測マシン	X から Y を弾き出す数式。
傾き (β_1)	交換レート	X が 1 増えると Y がどれだけ増えるか。
残差	削りカス	モデルが説明しきれなかったノイズ。
決定係数 (R^2)	テストの点数	モデルがどれくらい優秀かのスコア (0-1)。

第 12 回重回帰分析と因果推論

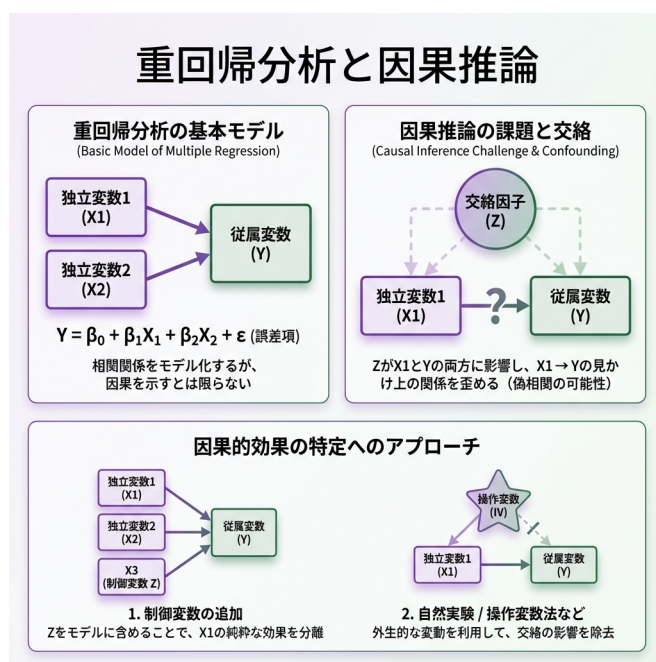


Figure12: 重回帰分析と因果推論

複雑な現実を解き明かす

「勉強すれば成績は上がる」と言うが、成績に影響するのは勉強時間だけだろうか？ 地頭の良さ、塾の有無、親の収入、睡眠時間.....現実は無数の要因が絡み合っている。

たった一つの原因で結果が決まることなど、世の中にはほとんどない。だからこそ、複数の要因を同時に扱う **重回帰分析** が必要になる。

今回は、絡み合った糸をほぐし、真の原因（因果関係）を見つけ出すための探偵技術を学

ぼう。

1. 複数のツマミを操る：重回帰分析

データ分析を「ミキシングコンソール」の操作だと想像してほしい。単回帰分析は「ボリューム」という一つのツマミしか持たなかった。重回帰分析では、「ボリューム」「低音」「高音」など、複数のツマミ（説明変数）を同時に操作して、最適なサウンド（予測）を作り出す。

$$\text{結果 (Y)} = \text{切片} + (\beta_1 \times \text{要因 1}) + (\beta_2 \times \text{要因 2}) + \dots + \text{誤差}$$

「他を固定して」見る技術 (Ceteris Paribus)

重回帰分析の凄さは、「他の条件が同じならば」という仮定を数式の中で実現できる点にある。

例えば、チップ額 (Y) を支払額 (X_1) と客数 (X_2) で説明する場合：

係数 β_1 は、「客数が同じグループの中で比べたとき、支払額が 1 ドル増えるとチップはどうか」を表す。

あたかも実験室で条件を統制したかのように、純粋な効果を取り出せるのだ。

実践: 支払いと人数の分離

```
import pandas as pd
import statsmodels.formula.api as smf

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
tips = pd.read_csv(url)

# 重回帰モデル：チップを「支払額」と「人数」で説明する
```

```
model_multi = smf.ols("tip ~ total_bill + size", data=tips).fit()
model_multi.summary()
```

Dep. Variable:	tip	R-squared:	0.468
Model:	OLS	Adj. R-squared:	0.463
Method:	Least Squares	F-statistic:	105.9
Date:	Tue, 10 Feb 2026	Prob (F-statistic):	9.67e-34
Time:	22:35:53	Log-Likelihood:	-347.99
No. Observations:	244	AIC:	702.0
Df Residuals:	241	BIC:	712.5
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.6689	0.194	3.455	0.001	0.288	1.050
total_bill	0.0927	0.009	10.172	0.000	0.075	0.111
size	0.1926	0.085	2.258	0.025	0.025	0.361

Omnibus:	24.753	Durbin-Watson:	2.100
Prob(Omnibus):	0.000	Jarque-Bera (JB):	46.169
Skew:	0.545	Prob(JB):	9.43e-11
Kurtosis:	4.831	Cond. No.	67.6

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

解釈: - total_bill: **0.093** - 「もし人数が同じなら」、支払いが1ドル増えるごとにチップは約9.3セント増える。 - size: **0.193** - 「もし支払額が同じなら」、人数が1人増えるごとにチップは約19.3セント増える。

2. 実践的モデリング

現実には数値データだけではない。「性別」や「曜日」といった文字データ（カテゴリ変数）も分析に使える。

ダミー変数：スイッチの ON/OFF

「喫煙席か？」のような情報は、計算できるよう 0 (No) と 1 (Yes) に変換する。これをダミー変数と呼ぶ。

```
# 喫煙者ダミー (Yes/No) を追加
model_dummy = smf.ols("tip ~ total_bill + size + C(smoker)", data=tips).fit()

import pandas as pd
pd.DataFrame({"term": model_dummy.params.index, "estimate": model_dummy.params.v
```

	term	estimate
0	Intercept	0.709016
1	C(smoker)[T.Yes]	-0.083433
2	total_bill	0.093888
3	size	0.180332

`C(smoker)[T.Yes]` の係数は、喫煙席に座ることで生じる「上乗せ分」を表す（基準カテゴリとの差）。

交互作用：「相乗効果」に気づく

「ビールと枝豆」のように、組み合わせることで効果が変わることもある。例えば、「喫煙者の方が、支払額に対するチップの払いっぷりが良い（傾きが急）」かもしれない。

: は交互作用のみ、* は主効果+交互作用

```
model_inter = smf.ols("tip ~ total_bill * C(smoker)", data=tips).fit()
model_inter.summary()
```

Dep. Variable:	tip	R-squared:	0.506
Model:	OLS	Adj. R-squared:	0.500
Method:	Least Squares	F-statistic:	81.95
Date:	Tue, 10 Feb 2026	Prob (F-statistic):	1.56e-36
Time:	22:35:53	Log-Likelihood:	-338.91
No. Observations:	244	AIC:	685.8
Df Residuals:	240	BIC:	699.8
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.3601	0.202	1.782	0.076	-0.038	0.758
C(smoker)[T.Yes]	1.2042	0.312	3.856	0.000	0.589	1.819
total_bill	0.1372	0.010	14.172	0.000	0.118	0.156
total_bill:C(smoker)[T.Yes]	-0.0676	0.014	-4.762	0.000	-0.096	-0.040

Omnibus:	45.973	Durbin-Watson:	2.078
Prob(Omnibus):	0.000	Jarque-Bera (JB):	122.757
Skew:	0.830	Prob(JB):	2.21e-27
Kurtosis:	6.053	Cond. No.	132.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

total_bill:C(smoker)[T.Yes] が有意なら、2つの要因には相乗効果（あるいは相殺効果）があると言える。

3. 真犯人を探せ：因果推論とバイアス

「相関関係があっても因果関係とは限らない」。耳にタコができるほど聞いた言葉だが、その正体は何だろうか？ 最大の原因は、**交絡因子**（こうらくいんし）という「裏で糸を引く黒幕」の存在だ。

典型例：教育と賃金のミステリー

「高学歴な人は給料が高い」というデータがある。しかし、これは純粋に教育のおかげだろうか？ もしかすると、「元々の能力が高い人」が「高学歴になりやすく」、かつ「給料も高い」だけかもしれない。

この「能力」こそが交絡因子だ。これを無視して分析すると、教育の効果を過大評価してしまう（**除外変数バイアス**）。

シミュレーション：バイアスの除去

重回帰分析を使えば、この黒幕（交絡因子）を表舞台に引きずり出し、無力化できる。ここでは、あえて「神様の視点」で真実のデータを作り出し、それを分析者が正しく見抜けるか実験してみよう。

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf

rng = np.random.default_rng(123)
n = 200

# 1. 神様の視点（データの生成）
ability = rng.normal(100, 15, size=n) # 能力（黒幕）
education = 12 + 0.05 * ability + rng.normal(0, 2, size=n) # 能力が高いと教育年数も
wage = 20 + 2 * education + 0.3 * ability + rng.normal(0, 5, size=n) # 賃金は教育
```

```
sim_data = pd.DataFrame({"wage": wage, "education": education, "ability": ability})

# 2. 失敗：黒幕（能力）を無視してしまう（除外変数バイアス）
m1 = smf.ols("wage ~ education", data=sim_data).fit()
m1.params

# 3. 成功：黒幕をモデルに入れてコントロール（統制）する
m2 = smf.ols("wage ~ education + ability", data=sim_data).fit()
m2.params
```

```
Intercept    22.070785
education     1.799787
ability       0.311450
dtype: float64
```

能力をモデルに加える（コントロールする）ことで、教育が賃金に与える純粋な効果（係数）は、真の値である **2.0** に近づいた。重回帰分析は、単なる予測ツールではない。変数を適切に選ぶことで、偏りのない**因果推論**を行うための強力な武器になるのだ。

4. 良いモデルの選び方

変数を増やせば増やすほどモデルは現実には合致していくが、やりすぎは禁物。

多重共線性（マルチコ）：似た者同士の喧嘩

似たような変数（例：身長と座高）を両方入れると、お互いが邪魔をし合って、係数の推定がおかしくなる。**VIF** という指標チェックし、10 を超えていたら要注意。

```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = model_multi.model.exog
names = model_multi.model.exog_names
```

```
vif = pd.DataFrame(
    {"term": names, "VIF": [variance_inflation_factor(X, i) for i in range(X.shape[0])]}
)
vif.query('term != "Intercept"')
```

	term	VIF
1	total_bill	1.557586
2	size	1.557586

調整済み決定係数：シンプルさへのボーナス

通常の決定係数 R^2 は、無駄な変数を足しても増えてしまうという欠点がある。そこで、「変数の数」が増えることにペナルティを課した**調整済み決定係数 (Adjusted R^2)** を使おう。この値が大きいほど、バランスの良い優れたモデルと言える。

```
# 調整済み決定係数 (Adjusted R-squared) を比較してみよう
model_simple = smf.ols("tip ~ total_bill", data=tips).fit() # 前回の単回帰モデル

model_simple.rsquared_adj
model_multi.rsquared_adj
model_dummy.rsquared_adj
```

```
0.46203699058834746
```

モデルが複雑になるにつれて、値が改善しているのがわかるはずだ。（※より高度な基準として AIC や BIC もあるが、まずはこれを羅針盤にしよう）

係数の可視化

数字の羅列を見るよりも、グラフで可視化したほうが直感的に理解できる。

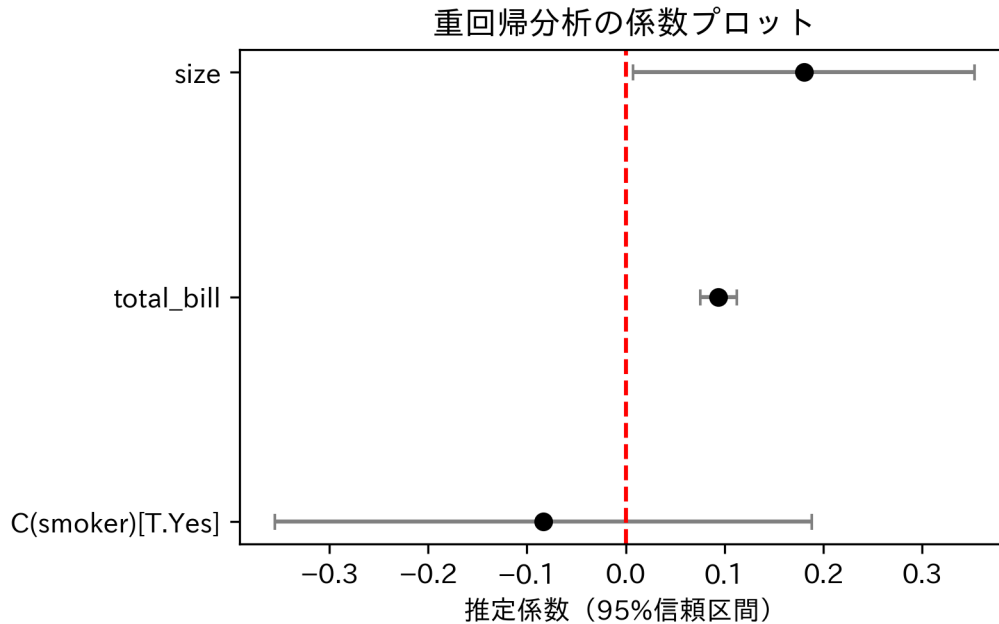

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib # noqa: F401

coef = pd.DataFrame(
    {
        "term": model_dummy.params.index,
        "estimate": model_dummy.params.values,
    }
)

conf = model_dummy.conf_int().rename(columns={0: "conf_low", 1: "conf_high"})
coef = coef.merge(conf, left_on="term", right_index=True)
coef = coef.query('term != "Intercept"]').sort_values("estimate").reset_index(drop=True)

y = np.arange(len(coef))
xerr = np.vstack([coef["estimate"] - coef["conf_low"], coef["conf_high"] - coef["estimate"]])

plt.errorbar(coef["estimate"], y, xerr=xerr, fmt="o", color="black", ecolor="gray", capsize=5)
plt.axvline(0, color="red", linestyle="--")
plt.yticks(y, coef["term"])
plt.title("重回帰分析の係数プロット")
plt.xlabel("推定係数 (95% 信頼区間) ")
plt.tight_layout()
plt.show()
```



最終課題

自分の興味のあるデータセットを見つけ、分析レポートを作成してください。データセットの例： - mpg（自動車の燃費データ：seaborn の例として有名） - diamonds（ダイヤモンドの価格データ：データセットとして定番） - palmerpenguins（ペンギンの身体測定データ：Python でも利用可能）

レポート構成案:

1. **問いの設定:** 何を明らかにしたいか（例：ダイヤモンドの価格を決めるのはカラット数か、透明度か？）
2. **データの可視化:** ヒストグラムや散布図でデータを「見る」。
3. **モデリング:** 重回帰モデルを構築し、係数を推定する。
4. **結果の解釈:** 「他の条件を一定としたとき」、その変数は結果にどう影響しているか？ を言葉で説明する。

講義のまとめ

本講義では、データの読み込みから可視化、基本的な統計的推測、そして回帰分析による因果へのアプローチまでを学びました。これらはデータサイエンスの基礎体力となるスキルです。さらに深く学びたい方は、計量経済学や機械学習のコースへ進むことを推奨します。

練習問題

1. **モデル選択の実践:** tips データセットにおいて、チップ額 (tip) を説明する変数 (例: total_bill, size, time, smoker など) の組み合わせを、調整済み R^2 や AIC を用いて探してください。
2. **交互作用の解釈:** 先ほどの喫煙者と支払額の交互作用モデルについて、具体的な数値を用いて、「喫煙者なら 10 ドル増えるごとにチップはどう変わるか」を説明してください。
3. **交絡の発見:** あなたの身の回りで「相関はあるが因果ではない (共通の原因がある)」と思われる例を挙げ、因果図を描いて説明してください。